

PoVGEN: Generating Proof of Vulnerabilities via Neural-Symbolic Execution

Abstract—Software vulnerabilities are a persistent threat, but reproducing them is often difficult due to the scarcity of public Proof-of-Vulnerability (PoV), even when patches are available. To address this, we introduce PoVGEN, a framework whose technical core is a set of fine-tuned LLMs specialized for the formal sub-tasks of PoV generation. PoVGEN decomposes PoV generation into a three-step pipeline: (1) Vulnerability Manifestation Localization pinpoints the vulnerability manifestation point using a fine-tuned semantics-aware LLM; (2) Path-Sensitive Reachability Exploration identifies viable execution paths from a program entry to the manifestation point on an interprocedural control-flow graph; and (3) Neural-Symbolic PoV Generation employs two fine-tuned LLMs that extract and solve path constraints, transferring formal behavior from a traditional symbolic execution engine and SMT solver into the models. Our evaluation shows that PoVGEN successfully generates PoVs for 80.20% of the vulnerabilities in a recent dataset, outperforming traditional fuzzers (up to 50.20%), a symbolic execution engine (2.45%), and LLM-based approaches (up to 35.89%). More importantly, PoVGEN generated valid PoVs for 75.09% of 269 real-world CVEs that lack public PoVs, and reproduces 70.20% of vulnerabilities even without patch information. The fine-tuned models match frontier commercial LLMs on the formal sub-tasks at over $100\times$ lower per-sample cost. Applying the generated PoVs identified six flawed patches in disclosed CVEs (all subsequently fixed) and discovered five previously unknown vulnerabilities, four of which have been confirmed and fixed by the developers.

I. INTRODUCTION

Software vulnerabilities pose a significant threat to the cybersecurity [1]. Their prevalence is widespread across software ecosystems [2], [3], [4], [5], [6]. Consequently, timely and effective vulnerability patching is a cornerstone of cyber defense [7], [8], [9]. However, a patch alone is insufficient. To validate a vulnerability’s impact or a patch’s correctness, one needs a concrete *Proof-of-Vulnerability (PoV)*—an input that triggers the vulnerability in a program [10].

A PoV turns a vulnerability from an abstract warning into a concrete, reproducible security condition. It validates issues for vendors, provides actionable test inputs for defenders, and removes ambiguity for researchers. It is also essential for verifying patches: roughly 7% of patches are incomplete and 5% regressive [11], and a working PoV lets defenders confirm whether a patch actually closes the bug [12], [13].

However, this critical need for PoVs is starkly contrasted with their scarcity. As our recent study in Figure 1 reveals, while most (73.0%) of the disclosed vulnerabilities have patches available, only a small fraction (2.42%-16.80%) of them come with available PoVs, making it urgent to generate the missing PoVs for these vulnerabilities.

Efforts to address this PoV scarcity exist. For example, the ARVO dataset [10] takes a crucial step by curating reproducible vulnerabilities from platforms like OSS-Fuzz [14]. However, its scope is limited to vulnerabilities already discovered in the OSS-Fuzz ecosystem. It cannot generate PoVs for the broader set of CVEs which are not found by OSS-Fuzz tools. Other existing automated PoV generation techniques fall short. Traditional methods like fuzzing are often random and time-consuming for reproducing a specific, known bug [15]. Other approaches like symbolic execution [16] are powerful but struggle to scale to large, real-world projects. Other tools target narrow classes of vulnerabilities, such as memory corruption [17], object injection [18], [19], or specific execution environments like kernels [20] and web applications [21].

Recent advancements have explored using large language models (LLMs) for similar tasks. PoCGen [22] combines LLMs with static and dynamic analysis in a synthesis-and-validate loop, asking the LLM to produce an exploit and refining the prompt from execution feedback. However, the technique places the entire synthesis on the LLM and relies on it to navigate path and constraint reasoning implicitly, which is unlikely to scale to disclosed CVEs whose triggering conditions involve precise reasoning over execution paths and input constraints. The closest line of work uses LLMs to drive symbolic or concolic execution: AutoBug [23] uses LLMs as path constraint solvers for symbolic execution, but does not generate concrete inputs. ConcoLLMic [24] and CottonTail [25] use LLMs to drive concolic execution and produce inputs, but, like all concolic approaches, they require concrete seed inputs to begin exploration. CottonTail can also generate seeds via LLM, but the resulting exploration targets coverage of a parser, not a specific disclosed vulnerability. For vulnerabilities whose triggering inputs are not public, none of these techniques is directly applicable: there are no seed inputs available, and coverage-driven exploration provides no mechanism to direct the analysis to the specific vulnerability. Generating PoVs for vulnerabilities requires a technique that can identify and reach the specific manifestation of a vulnerability, derive the constraints that drive execution to it, and solve those constraints to produce a concrete input.

Therefore, we present PoVGEN, a framework whose technical core is a set of fine-tuned LLMs specialized for the formal sub-tasks of PoV generation. PoVGEN operates in two modes—a patch-guided mode that uses the patch as a precise

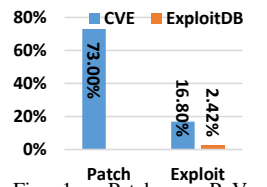


Fig. 1. Patch availability.

anchor, and a `patch-free` mode that operates without patch information for zero-day scenarios—and decomposes PoV generation into three steps. First, in *Vulnerability Manifestation Localization*, a fine-tuned semantics-aware LLM pinpoints the vulnerability manifestation point via a lightweight interprocedural analysis. Second, in *Path-Sensitive Reachability Exploration*, a guided search on an interprocedural control-flow graph identifies viable execution paths from a program entry to the manifestation point, reducing a massive project to a small set of focused traces. Third, in *Neural-Symbolic PoV Generation*, two fine-tuned LLMs extract and solve path constraints on those traces, transferring formal behavior from a traditional symbolic execution engine and SMT solver into the LLMs.

We fine-tune local open-source models rather than prompt frontier commercial LLMs because constraint extraction and PoV synthesis exhibit a distributional structure—path-specific SMT-LIB formulas paired with C execution paths and concrete byte-level inputs—that is distinct from the natural code distribution frontier models are pretrained on. Closing this gap requires specialization, and fine-tuning is the mechanism by which traditional symbolic tools transfer their formal behavior into the models. Besides, the cost structure of PoV generation makes API-based approaches impractical at scale. Constraint extraction and PoV synthesis run on every explored path of every target CVE, consuming several million tokens per sample on average—over \$1 per sample even on cost-optimized commercial models, and several dollars on frontier models. Fine-tuning amortizes a one-time training cost into zero marginal cost per inference.

Our evaluation demonstrates that POVGEN is effective and practical. On the ARVO benchmark, POVGEN reproduces 80.20% of vulnerabilities in the `patch-guided` mode, outperforming state-of-the-art fuzzers (max 50.20%), a traditional symbolic execution engine (2.45%), and LLM-based PoV generation approaches (max 35.89%). The advantage over fuzzers and symbolic execution validates the search-space narrowing through manifestation localization and path-sensitive exploration. The advantage over LLM-based approaches validates the fine-tuning choice in neural-symbolic PoV generation: a direct evaluation of the fine-tuned models shows that POVGEN’s constraint extraction model reaches 89.63% semantic recall, compared to 57.04% for the same base open-source model without fine-tuning, and 86.14% for Claude-4-Sonnet at \$2.06 per sample versus \$0.02 per sample for POVGEN. Most significantly, on our primary use case—reproducing CVEs that lack public PoVs—POVGEN generated valid PoVs for 75.09% of 269 vulnerabilities.

Beyond reproducing disclosed CVEs, the generated PoVs also enabled patch validation and zero-day discovery: we identified six flawed patches in disclosed CVEs, all of which have been confirmed fixed in later versions, and discovered five previously unknown vulnerabilities, four of which have been confirmed and fixed by upstream maintainers, with one CVE ID assigned. Even without patch information (`patch-free` mode), POVGEN achieves a 70.20% success rate, demon-

```
diff a/.../stun_msg.c b/.../stun_msg.c
2360c2368
< if (pdu_len < attr_val_len) {
...
> if (pdu_len < attr_val_len + sizeof(pj_stun_attr_hdr)) {

ERROR: AddressSanitizer: heap-buffer-overflow on address
...
READ of size 1 at 0x60300000002a thread T0
#0 0x4eb0cb in decode_errcode_attr .../stun_msg.c:1760:22
#1 0x4e4902 in pj_stun_msg_decode .../stun_msg.c:2446:22
#2 0x4dd275 in stun_parse .../fuzz-stun.c:43:14
#3 0x4dd275 in LLVMFuzzerTestOneInput .../fuzz-stun.c:85:11
#4 0x531b8a in ExecuteFilesOnlyByOne .../afpp_driver.c:234:7
#5 0x53195c in main .../afpp_driver.c:318:12
#6 0x7841ab18c082 in __libc_start_main
#7 0x41e6bd in _start (/out/fuzz-stun+0x41e6bd)
```

Fig. 2. An example of patch and sanitizer output for the CVE-2022-23537 vulnerability from the PJSIP project.

strating its applicability to newly discovered vulnerabilities. POVGEN’s modular design also supports extension to other vulnerability types and programming languages, and its individual components are usable as standalone tools for symbolic execution and vulnerability manifestation localization.

We make the following contributions:

- We design a novel automatic PoV generation technique that combines a fine-tuned vulnerability manifestation localization model, path-sensitive reachability exploration and fine-tuned neural-symbolic PoV generation models (§III).
- We evaluate POVGEN on 490 vulnerabilities from the ARVO benchmark and 269 real-world CVEs that lack public PoVs, achieving 80.20% and 75.09% reproduction rates respectively, outperforming state-of-the-art fuzzers, symbolic execution, and LLM-based baselines (§V-B–§V-C).
- We use POVGEN to identify six flawed patches (all subsequently fixed) and to discover five previously unknown vulnerabilities (four confirmed and fixed by upstream maintainers), demonstrating practical security utility beyond CVE reproduction (§V-F).
- We release POVGEN as an open-source artifact, including the fine-tuned model weights, fine-tuning datasets, and evaluation pipeline, enabling reproducible PoV generation research.

II. BACKGROUND AND MOTIVATION

This section discusses the background and motivation for disclosed vulnerability PoV generation along with the challenges and insights for POVGEN design.

A. CVEs and PoVs: A Motivating Study

The Common Vulnerabilities and Exposures (CVE) system [26] records publicly disclosed vulnerabilities. Although patches are frequently available, concrete PoVs—inputs that reliably trigger the vulnerability—are often missing. To quantify this gap, we analyzed 7,474 CVEs related to open-source projects from 2012 to 2024. As shown in Figure 1, 73.0% have patches but only 16.8% have public PoVs. The ExploitDB database shows the same pattern: among 5,412 CVEs between 2012-2024, only 131 (2.42%) provide PoVs. This gap hinders verification of vulnerabilities and their fixes, highlighting the need for automated PoV generation.

B. Challenges and Insights

Existing techniques remain inadequate for this task: traditional fuzzing is not cost-effective for known bugs [15], symbolic execution struggles with scalability [16], and other tools are narrowly focused on specific vulnerability classes [17] or environments [20]. While LLMs have shown promise in code-related tasks [7], [27], [28], they also face challenges when applied to this complex domain. To demonstrate these challenges, we use a real-world vulnerability from Figure 2 as a running example, and discuss them as three interleaved challenge-insight pairs.

- **Challenge 1: Large Search Space for PoV Generation.**

A fundamental challenge in symbolic PoV generation is the size of the search space: only a small fraction of program paths reach the conditions that trigger the vulnerability. Without a concrete target to direct exploration, an analysis enumerates paths and quickly drowns in irrelevant traces, especially in real-world projects with thousands of functions. **Insight 1: Narrow the Search Space with the Manifestation Point.** POVGEN narrows the search space by directing exploration toward the vulnerability manifestation point, recovered from the source code (and patch, when present) via a fine-tuned semantics-aware LLM. Recovering this point is non-trivial because patches frequently do not pinpoint where the vulnerability triggers [29]: in Figure 2, the patch for CVE-2022-23537 is at line 2360 in `stun_msg.c`, but the bug manifests at line 1760. With this point as an anchor, path exploration becomes a focused reachability problem from the program entry to the manifestation point, reducing a project-wide search to a small set of traces.

- **Challenge 2: Constraint Identification and Solving.** Even on focused traces, generating a PoV requires deriving the path constraints that drive execution to the manifestation point and solving them for concrete inputs. Both subtasks are difficult. Constraint extraction must track data flow through pointers, structures, and inter-procedural calls accurately enough to reflect the program’s actual semantics, and the resulting constraints often involve symbolic memory, nonlinear arithmetic, or string operations that strain traditional SMT solvers. As shown in Figure 2, the path to the buffer over-read in `stun_msg.c` traverses multiple functions and accumulates constraints on the input bytes’ lengths, types, and offsets that a general-purpose solver does not handle uniformly.

Insight 2: Combine Symbolic Reasoning with LLM Code Understanding. POVGEN addresses Challenge 2 by combining traditional symbolic tools with LLMs trained on their behavior. We fine-tune two specialized LLMs: one to extract SMT constraints from a given execution path, and one to synthesize a satisfying PoV from those constraints. The fine-tuning data is generated by a traditional symbolic execution engine and SMT solver on auxiliary programs, transferring their formal behavior into the LLMs while letting the LLMs apply their code understanding to constructs that traditional tools handle poorly. A traditional SMT solver remains as

a fallback for cases where the LLM produces an invalid solution, preserving the soundness guarantees of symbolic execution where they apply.

- **Challenge 3: Whole-Project Scale and Formal Sub-Task Specialization.** The two LLMs from Insight 2 must run on every focused trace produced by Insight 1, and a real-world project produces many such traces. The PJSIP project in Figure 2, for instance, contains 106.8k lines of code, and reproducing a single CVE in our evaluation consumes 4.57M tokens on average across all explored paths. At this scale, two concerns compound. First, constraint extraction and PoV synthesis exhibit a distributional structure—path-specific SMT-LIB formulas paired with C execution paths and concrete byte-level inputs—that is distinct from the natural code distribution frontier models are pretrained on, so general-purpose LLMs are not aligned with the formal sub-tasks out of the box. Second, frontier commercial models (e.g., Claude-4-Sonnet, OpenAI-o3) are expensive at this token volume, while open-source models are inexpensive but underperform on these sub-tasks without specialization.

Insight 3: Fine-Tune Local Open-Source Models on Traditional-Tool Traces. POVGEN addresses Challenge 3 by fine-tuning local open-source models on data generated by traditional symbolic tools (Insight 2). The distributional gap between the formal sub-tasks and frontier pretraining requires specialization, and fine-tuning is the mechanism by which traditional symbolic tools transfer their formal behavior into the models. This brings open-source models to match or exceed frontier commercial models on these sub-tasks (§V-D). Running fine-tuned models locally also amortizes a one-time training cost into zero marginal cost per inference, avoiding the per-path API spend that would otherwise dominate the budget at scale.

III. APPROACH

We present our technique, starting with the design overview followed by details on each component.

A. Overview

The overall design of POVGEN is depicted in Figure 3. We target a realistic scenario for automatically reproducing a disclosed vulnerability for which the source code, patch (optional), and program entries are available. To support the challenging task of PoV generation, POVGEN consists of three phases: Fine-Tuning Data Preprocessing, Fine-Tuning, and PoV Generation.

Phase 1: Fine-Tuning Data Preprocessing. We first prepare datasets to fine-tune specialized LLMs. We choose fine-tuning rather than prompting general-purpose LLMs because PoV generation requires strict adherence to manifestation localization, logical constraints, and SMT syntax, where general models often hallucinate, while fine-tuning aligns the models with these formal requirements. We prepare data through two parallel pipelines. For vulnerability manifestation localization, POVGEN performs on-demand interprocedural slicing on the source code to generate focused vulnerability slices, which together with the ground-truth patches and manifestation

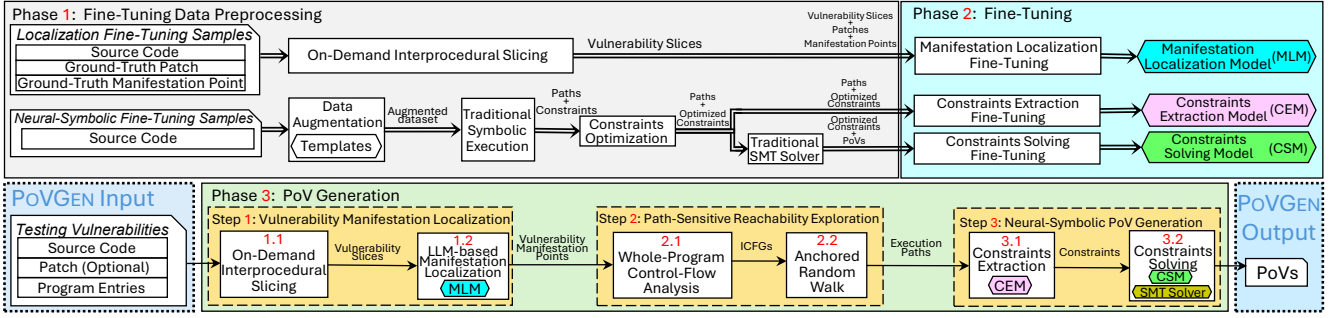


Fig. 3. An overview of POVGEN’s design, including its inputs, three main working phases, steps, and outputs.

points form the fine-tuning data for the Manifestation Localization Model (MLM). For neural-symbolic PoV generation, we augment the source code with predefined templates to introduce complex control flows and data structures (e.g., loops, pointers) often missing in standard datasets, ensuring the models can generalize to real-world software. A traditional symbolic execution engine then generates path constraints, which are optimized to form the dataset for the Constraints Extraction Model (CEM); a traditional SMT solver solves these constraints to produce PoVs for fine-tuning the Constraints Solving Model (CSM).

Phase 2: Fine-Tuning. POVGEN leverages the preprocessed datasets to fine-tune the three specialized LLMs that form the core of its analysis pipeline. We employ a modular design with three separate models to decompose the PoV generation task into sub-problems—localization, extraction, and solving—allowing each model to specialize without interference. The MLM is fine-tuned on slices, patches, and manifestation points to pinpoint the statement where the vulnerability manifests. The CEM is fine-tuned on optimized constraints and their corresponding paths to derive the SMT constraints from an execution path. The CSM is fine-tuned on constraint-PoV pairs to act as a neural SMT solver, generating a PoV that satisfies the given constraints. We separate CEM and CSM to mirror the distinction between symbolic execution engines and SMT solvers; CEM focuses on mapping code semantics to logical formulas, while CSM specializes in the mathematical reasoning required to satisfy them.

Phase 3: PoV Generation. This phase takes a testing vulnerability as input. Step 1 performs on-demand interprocedural slicing on the target code and feeds the slices into the fine-tuned MLM to localize the manifestation point. We use on-demand slicing to incrementally capture necessary interprocedural dependencies while keeping the input size tractable for the LLM, avoiding the scalability issues of whole-program analysis. Step 2 constructs an Interprocedural Control-Flow Graph (ICFG) and performs an Anchored Random Walk—using the program entry, optional patch location, and manifestation point as anchors—to discover viable execution paths. This guided search narrows the search space (Challenge 1) by isolating specific, viable traces. Step 3 uses the fine-tuned CEM to derive SMT constraints for each path, which are solved by the CSM (with a traditional SMT solver fallback) to generate the PoV. This neural approach decouples constraint extraction from full-program symbolic

execution, allowing us to analyze complex paths that would cause traditional engines to time out.

To accommodate different scenarios, POVGEN operates in two modes. In the *patch-guided* mode, the patch serves as a semantic anchor, enabling precise manifestation localization and a focused, multi-segment search (Entry $\rightarrow$ Patch $\rightarrow$ Manifestation). In the *patch-free* mode, POVGEN relies solely on the source code, predicting the manifestation point directly and exploring paths from the entry point. This mode acts as a fallback, trading off some analysis effectiveness for the ability to handle unpatched, zero-day vulnerabilities.

B. Fine-Tuning Data Preprocessing (Phase 1)

Before the main PoV generation pipeline can operate, POVGEN first prepares datasets to fine-tune specialized LLMs. This preprocessing phase is crucial for tailoring the models to their specific, complex tasks. It involves two parallel pipelines to generate data for manifestation localization and neural-symbolic PoV generation, respectively.

1) *Localization Fine-Tuning Data Preparation:* To identify vulnerability manifestation points to serve as the exploration targets, we first fine-tune the manifestation localization model (MLM). The overall process is shown in Algorithm 1. It first conducts **on-demand interprocedural slicing** on the fine-tuning samples (D_{ft}) to generate focused code slices (Lines 4–6) with the helper function *OnDemandSlicing* (Lines 9–15). These slices, along with the ground-truth patches and manifestation points (MPs), are then used for **manifestation localization fine-tuning** to get the *manifestation localization model MLM* (Line 7).

On-Demand Interprocedural Slicing. To identify vulnerability manifestation points, we build the searching scope by tracing the data and control flows in the program. For fine-tuning data where ground-truth patches are available, we leverage the fact that the manifestation point depends on the patch and generate an interprocedural slice starting from the patch.

We first construct a system dependency graph (SDG) for each program through interprocedural dependency analysis. An SDG composes the individual Program Dependence Graphs (PDGs) of each function with edges that represent interprocedural control and data flow dependencies [30], [27], serving as the basis for the subsequent slicing.

Inputting a whole-program slice to the MLM is infeasible due to the model’s input size limit. We therefore conduct

Algorithm 1: Fine-Tuning Manifestation Localization Model (MLM)

Input: D_{ft} : Localization fine-tuning samples
Output: MLM : Manifestation Localization Model (MLM)

```
1 Function MLMFineTuning( $D_{ft}$ )
2    $Progs, Patches, MPs \leftarrow D_{ft}$ 
3    $Slices \leftarrow []$ ;
4   foreach  $P_i$  in  $Progs$  do
5      $S_i \leftarrow \text{OnDemandSlicing}(P_i)$ ;
6      $Slices.insert(S_i)$ ;
7    $MLM \leftarrow \text{FineTune}(Slices, Patches, MPs)$ ;
8   return  $MLM$ ;

9 Function OnDemandSlicing( $P$ )
10   $SDG \leftarrow \text{ConstructSDG}(P)$ ;
11   $S \leftarrow \text{InitialSlice}(SDG)$ ;
12  for  $i \leftarrow 1$  to  $MAX\_ITERATIONS$  do
13     $F_{context} \leftarrow \text{QueryLLMForContext}(S)$ ;
14     $S \leftarrow \text{SU ExpandSlice}(SDG, S, F_{context})$ ;
15  return  $S$ ;
```

on-demand interprocedural slicing, starting at the patching function (or program entries when the patch is unavailable) and performing forward and backward slicing to retain only statements with a data or control dependency on the patching statements. To provide context beyond the patching function, we prompt a general-purpose LLM with the current slice to identify additional relevant context functions, leveraging the LLM’s ability to understand code semantics [31].

The expanded slice is fed back to the LLM for further rounds of analysis until a threshold is reached (Lines 12-14 in Algorithm 1). This gradual expansion balances coverage with tractability, incrementally introducing relevant dependencies without exceeding context windows. We empirically set the threshold to three based on statistics on InterPVD [29] and ARVO [10], observing that most manifestation-relevant functions are discoverable within three iterations.

After slicing, the triplets of slices, ground-truth patches, and ground-truth manifestation points form the fine-tuning data for Phase 2.

2) *Neural-Symbolic Fine-Tuning Data Preparation*: To effectively extract and solve the constraints to generate PoVs, we fine-tune two separate models for constraints extraction and constraints solving, respectively. The overall process, detailed in Algorithm 2, fine-tunes two specialized LLMs with knowledge from a traditional symbolic execution engine. This involves augmenting the existing dataset with pre-defined templates (Lines 2-3), generating constraints and PoVs from the augmented dataset (Lines 5-13) and using them to fine-tune a constraints extraction model (CEM) and a constraints solving model (CSM) (Lines 14-15).

Data Augmentation. We start by collecting fine-tuning samples from existing static analysis benchmarks, which symbolic execution engines can easily process to generate constraints and PoVs. These benchmarks already cover a wide range of vulnerability patterns and provide most of the real-world complexity in our fine-tuning data. However, they underrepresent a few control- and data-flow motifs that frequently appear in real-world CVEs, such as loops with

symbolic bounds and pointer arithmetic with symbolic offsets. Without these motifs, the model is hard to prepare for the path-level reasoning required at PoV generation time.

To complement the benchmarks, we add four templates (Figure 4) that inject these patterns. The templates do not aim to cover the full diversity of real-world CVEs; that diversity comes from the benchmark samples. Their role is narrower: to expose the model to (1) *loops with symbolic bounds*, (2) *pointer and symbolic dereference*, (3) *symbolic array and nested loops*, and (4) *pointer arithmetic with a symbolic offset*. These four patterns are chosen because they are common in software vulnerabilities and are the patterns most likely to cause path explosion, memory aliasing, and complex constraints during symbolic execution. Together, the benchmark samples and the templates prepare the LLMs to handle the control- and data-flow patterns inherent in real-world vulnerabilities.

Traditional Symbolic Execution. We then run a traditional symbolic execution engine on the augmented source code. Key variables that impact vulnerability triggering, declared at the beginning of the main functions, are marked as symbolic (e.g., the loop bound n , the array arr , or the pointer offset in the templates). For every path from the program entry to an exit point, the engine accumulates a set of logical constraints on the initial symbolic variables.

Constraints Optimization. While the raw constraints from the symbolic execution engine are logically correct, they are often not ideal for fine-tuning because of redundant expressions (e.g., trivially true clauses like $x == x$). Therefore, we perform a constraints optimization with the following rules:

- **Constant Folding.** We replace sub-expressions with only constant operands to one value (e.g., $(+ 2 2)$ becomes 4).
- **Algebraic Simplification.** We apply mathematical identities to simplify expressions (e.g., $(*x 1)$ simplifies to x).
- **Logical Simplification.** We reduce boolean expressions (e.g., $(\text{and } p \text{ true})$ simplifies to p).
- **Canonical Reordering.** We reorder terms within commutative expressions into a consistent, canonical form (e.g., $(+ y x)$ is always rewritten as $(+ x y)$).

Traditional SMT Solving. For each set of valid path constraints, the engine queries an SMT solver to find a concrete set of input values that satisfy the constraints. This satisfying assignment serves as a Proof-of-Vulnerability (PoV), a concrete test case that can trigger the specific path.

After the steps above, the collections of (paths, optimized constraints) and (optimized constraints, PoVs) form the fine-tuning data for the constraints extraction model (CEM) and constraints solving model (CSM), respectively.

C. Fine-Tuning (Phase 2)

With the datasets prepared in Phase 1, POVGEn proceeds to fine-tune its three specialized LLMs in the two pipelines. Each model is fine-tuned for a specific sub-task, transforming general-purpose LLMs into expert tools.

The three sub-tasks share a distributional structure—path-specific outputs in formal notations like SMT-LIB, paired with code execution paths and concrete inputs—that is distinct from

<pre>int main(){ int n; int sum = 0; for (int i = 0; i < n; i++){ if (i % 2 == 0) sum += i; assert (sum{op}{val}); } return 0; }</pre>	<pre>int main(){ int x; int *p = &x; assert (*p{op}{val}); return 0; }</pre>	<pre>int main(){ int arr[5]; int count = 0; for (int i = 0; i < 5; i++){ for (int j = i; j < 5; j++){ if (arr[i] + arr[j] > {val}) count++; } assert (count{op}{threshold}); } return 0; }</pre>	<pre>int main(){ int arr[3] = {1, 2, 3}; int offset; int *p = arr + offset; assert (offset >= 0 && offset < 3 && *p{op}{val}); return 0; }</pre>
---	--	---	--

Loop with a Symbolic Bound; Pointer and Symbolic Dereference; Symbolic Array and Nested Loops; Pointer Arithmetic with a Symbolic Offset;

Fig. 4. The templates used for data augmentation, where {op} is a random relational operator, {val} and {threshold} are random integers.

the natural code distribution frontier models are pretrained on. Closing this gap requires specialization, and fine-tuning is the mechanism by which traditional symbolic tools transfer their formal behavior into the models. Running fine-tuned models locally also amortizes a one-time training cost into zero marginal cost per inference.

1) *Manifestation Localization Fine-Tuning*: With the vulnerability slices, the next step is to fine-tune a manifestation localization model using the slices from the localization fine-tuning samples. The primary objective of this model is to identify the vulnerability manifestation statement within a given code slice. We fine-tune the manifestation localization model in a question-answer format, where the input is a code slice and the model is asked to pinpoint the vulnerability line.

The model is fine-tuned to predict the vulnerability manifestation statement, which we denote as the output Y . The fine-tuning data for this phase comprises all the slices obtained during the iterative on-demand slicing process described in §III-B1. To enhance the model’s robustness and mitigate false positives, we also incorporate negative samples: if a generated slice does not contain the ground-truth manifestation point, the expected output Y is explicitly set to `None`. This teaches the model to recognize and disregard irrelevant code segments. In case none of the iteratively generated slices covers the manifestation point, we create a ground-truth slice from the patch location to the manifestation point to ensure the model is exposed to a positive example.

The objective of the fine-tuning is to adjust the pretrained LLM’s weights, denoted by θ , to minimize the loss $\mathcal{L}$, which is the negative log-likelihood of generating the correct output Y given the input instruction I . This is formally expressed as: $\mathcal{L} = -\log P(Y|I, \theta)$.

2) *Neural-Symbolic Fine-Tuning*: With the refined datasets from Phase 1, we now proceed to fine-tune the two core models of our neural-symbolic PoV generation: the constraints extraction model (CEM) and the constraints solving model (CSM). This fine-tuning process augments the capabilities of traditional symbolic execution engines and SMT solvers to the LLMs, enabling them to perform these specialized tasks effectively.

The fine-tuning for both models is structured as an instruction-following task. For CEM, the goal is to extract path constraints from code. For CSM, the objective is to emulate an SMT solver by generating a concrete test case that satisfies a given set of constraints. We fine-tune two LLMs with specialized instructions: one to extract path constraints from code, and another to generate an input (PoV) satisfying those constraints. We follow the fine-tuning process in §III-C1 to

Algorithm 2: Neural-Symbolic Fine-Tuning

Input: D_{ft} : Neural-Symbolic Fine-Tuning Dataset
Output: CEM: Constraints Extraction Model,
 CSM: Constraints Solving Model

```
1 Function NeuralSymbolicFineTuning( $D_{ft}$ )
2    $D_{aug} \leftarrow \text{GenerateFromTemplates}()$ ;
3    $D_{neural} \leftarrow D_{ft} \cup D_{aug}$ ;
4    $Paths \leftarrow []$ ;
5    $Constraints \leftarrow []$ ;
6    $PoVs \leftarrow []$ ;
7   foreach  $P_i$  in  $D_{neural}$  do
8      $constr \leftarrow \text{SymbolicExecute}(P_i)$ ;
9      $constr_{opt} \leftarrow \text{Optimize}(constr)$ ;
10     $pov \leftarrow \text{TradSMTSolver}(constr_{opt})$ ;
11     $Paths.insert(P_i)$ ;
12     $Constraints.insert(constr_{opt})$ ;
13     $PoVs.insert(pov)$ ;
14   $CEM \leftarrow \text{FineTune}(Paths, Constraints)$ ;
15   $CSM \leftarrow \text{FineTune}(Constraints, PoVs)$ ;
16  return CEM, CSM;
```

adjust the pretrained LLM’s weights to minimize the loss for extracting and solving the constraints. After this fine-tuning process is complete, the CEM and CSM are prepared for the PoV Generation.

D. PoV Generation (Phase 3)

With the specialized models fine-tuned, POVGGEN is now ready to perform its primary task: automatically generating a PoV for a given vulnerability. This operational phase executes a systematic, three-step pipeline that leverages the fine-tuned models to generate concrete, working PoVs.

1) *Vulnerability Manifestation Localization (Step 1)*: We apply the fine-tuned MLM to each testing vulnerability. We first generate slices using the same on-demand interprocedural slicing process in §III-B1: in patch-guided mode, slicing starts from the patch; in patch-free mode, it starts from program entries. Each slice is fed into the MLM with the same instruction format used during fine-tuning, and the model outputs the predicted manifestation line. This serves as the anchor for the path exploration in Step 2, providing a target beyond the patch location and enabling a more focused search for viable execution paths.

2) *Path-Sensitive Reachability Exploration (Step 2)*: This step identifies viable execution paths from program entries to the manifestation point, passing through the patch location (when available). The overall process, detailed in Algorithm 3, begins with **whole-program control-flow analysis** to construct an interprocedural control-flow graph (ICFG) (Line 2). With the ICFG and the anchored points—the program entry (E), optionally the patch location (L_{patch}), and manifestation point ($L_{manifest}$)—it then repeatedly performs an anchored

Algorithm 3: Path-Sensitive Reachability Exploration

Input: P : Target program, E : Program entry, L_{patch} : Patch location (optional), $L_{manifest}$: Manifestation point
Output: $\mathcal{P}_{paths}$: A set of viable execution paths (exploration contexts)
Function PathSensitiveExploration($P, E, L_{patch}, L_{manifest}$)
1 $ICFG \leftarrow \text{ConstructICFG}(P)$;
2 $\mathcal{P}_{paths} \leftarrow []$;
3 **for** $i \leftarrow 1$ **to** $MAX_ITERATIONS$ **do**
4 $path \leftarrow \text{RandomWalk}(ICFG, E, L_{patch}, L_{manifest})$;
5 **if** $path$ is not null **then**
6 $\mathcal{P}_{paths}.\text{insert}(path)$;
7
8 **return** $\mathcal{P}_{paths}$;

Algorithm 4: Neural-Symbolic PoV Generation

Input: $C_{explore}$: Exploration contexts from Step 2, $desc$: Vulnerability description, $patch$: Patch (optional)
Output: $PoVs$: A set of generated Proof-of-Vulnerabilities
Function NeuralSymbolicPoVGeneration($C_{explore}, desc, patch$)
1 $PoVs \leftarrow []$;
2 **foreach** $path$ in $C_{explore}$ **do**
3 $constr \leftarrow \text{Predict}(CEM, path, desc, patch)$;
4 $pov \leftarrow \text{Predict}(CSM, path, desc, patch, constr)$;
5 **if** pov is not valid **then**
6 $pov \leftarrow \text{Solve}(SMTSolver, constr)$;
7 $PoVs.\text{insert}(pov)$;
8
9 **return** $PoVs$;

random walk for guided search to find paths $\mathcal{P}_{paths}$ (Lines 3-7). The discovered paths serve as contexts for the next step for the final PoV Generation (Line 8).

Whole-Program Control-Flow Analysis. We first conduct a *whole-program control-flow analysis* to build a complete interprocedural control-flow graph (ICFG), connecting the CFGs of individual functions by resolving all direct and indirect call sites. The resulting ICFG serves as the map for navigating from program entries to the manifestation point.

Anchored Random Walk. The next step is to find a viable execution path that demonstrates how the vulnerability can be triggered. The path must connect the available anchors: the program entry, the manifestation point, and the patch location (in patch-guided mode). The patch location is included in patch-guided mode because the patched statements are responsible for creating a vulnerable program state, so passing through them produces stronger exploration contexts for the next step. The execution path connecting these anchors provides the essential exploration context needed for the final PoV generation in Step 3.

We choose a guided random walk over full path enumeration because enumerating all interprocedural paths through specific nodes is infeasible due to path explosion and is undecidable in general. A purely random walk, on the other hand, would be inefficient on the vast state space of real-world programs. The anchored random walk strikes a balance: it preserves diversity through randomness while biasing exploration toward the anchors. The procedure has two phases.

Pre-computation phase. We run a backward breadth-first search (BFS) on the ICFG starting from each anchor that the walk needs to reach. In patch-guided mode, this is done for both the patch location and the manifestation point; in

TABLE I
VULNERABILITY TYPE DISTRIBUTION OF THE ARVO SAMPLES FOR RQ1 AND RQ2.

Vulnerability Type	#Samples	Vulnerability Type	#Samples
global-buffer-overflow	22	stack-buffer-underflow	3
heap-buffer-overflow	243	stack-use-after-return	3
heap-use-after-free	35	undefined-behavior	16
SEGV	55	use-after-poison	7
stack-buffer-overflow	31	use-of-uninitialized-value	75
<i>Total</i>	<i>490</i>		

patch-free mode, only for the manifestation point. The result is a distance map that records, for every reachable node in the ICFG, its shortest graph distance to the anchor. We use backward BFS because we need to know how far each node is from the target, not the other way around, and BFS is sufficient since ICFG edges are unweighted. These distance maps serve as the guidance signal during exploration.

Exploration phase. Starting from the program entry, the walk proceeds forward on the ICFG. At each branch, the next edge is selected randomly with probability inversely proportional to the successor’s pre-computed distance to the current anchor (specifically, weight $w = 1/(d + 1)$, where d is the distance), so closer nodes are preferred but more distant nodes can still be chosen. This keeps the walk biased toward the anchor while preserving diversity. In patch-guided mode, the walk is first guided by the distance to the patch location; once the patch is reached, the guidance switches to the manifestation point. In patch-free mode, the walk is guided directly toward the manifestation point. To ensure interprocedural validity, we maintain a call stack: function-call edges push a return identifier onto the stack, and a return edge is only taken if its identifier matches the top of the stack. This prevents the walk from returning to an arbitrary caller. The walk terminates when the manifestation point is reached, when a maximum number of attempts is exceeded, or when no valid successor exists.

To obtain multiple distinct paths, we run the walk repeatedly with different random seeds (Lines 4–7 in Algorithm 3). The complete pseudocode is given in Appendix B1. The resulting execution paths serve as the input for Step 3. We term these focused traces the “exploration context”; they bridge the gap between high-level program structure and the low-level details required for PoV generation.

3) *Neural-Symbolic PoV Generation (Step 3):* The final step extracts and solves the constraints to generate working PoVs, detailed in Algorithm 4. For each execution path, we provide the fine-tuned CEM with the path, the vulnerability description, and the ground-truth patch (if available) to derive the path constraints (Line 4). The fine-tuned CSM then solves these constraints in the same context to produce a satisfying PoV (Line 5). To handle PoVs that involve special characters, we instruct the CSM to “output the Python script to generate the PoV” rather than the PoV itself. For robustness, we incorporate a traditional SMT solver as a fallback (Lines 6-7, Figure 3): if the CSM fails on highly complex or corner-case constraints, the SMT2 constraints are passed to the solver.

TABLE II
CWE DISTRIBUTION OF CVE SAMPLES FOR RQ5.

CWE ID	#Samples	CWE ID	#Samples	CWE ID	#Samples
CWE-119	44	CWE-190	1	CWE-415	9
CWE-120	9	CWE-193	1	CWE-416	25
CWE-122	2	CWE-20	22	CWE-476	27
CWE-125	77	CWE-399	1	CWE-787	49
CWE-1284	1	CWE-401	1	<i>Total</i>	<i>269</i>

IV. IMPLEMENTATION

For vulnerability manifestation localization, we use Joern [32] to construct program dependence graphs and call graphs, and adapt SySeVR [33] for slicing. The necessary context functions are identified with Gemini-2.5-Flash, with a slicing depth of three, following prior work [29]. For path-sensitive reachability exploration, we compile the target program with WLLVM [34] and build interprocedural control-flow graphs using SVF [35]. The anchored random walk is implemented via SVF’s APIs, extracting up to 200 execution paths in parallel for efficiency.

Our choice of tools reflects stage-specific needs: Joern offers efficient slicing directly on source code for manifestation localization. For path-sensitive reachability exploration, we use SVF for more precise path analysis. To address potential scalability issues with whole-program analysis, we only employ SVF’s lightweight control-flow analysis module, avoiding its more heavyweight data-flow analysis. This choice represents a deliberate trade-off: the precision of the execution paths found is critical for the effectiveness of our final PoV generation. This approach, therefore, balances analysis efficiency with the path accuracy required for the subsequent steps to succeed.

For the CEM and CSM fine-tuning, we generate constraints and PoVs with symbolic execution engine KLEE [16] and SMT solver Z3 [36]. Model fine-tuning is performed with Unsloth [37], using Llama-3.2-3B for MLM and Llama-3.1-8B for CEM and CSM.

All experiments are conducted on a workstation with an AMD Threadripper Pro 5595WX 4.5GHz processor, four NVIDIA RTX A6000 GPUs, and 512GB of RAM, running Ubuntu 22.04.

V. EVALUATION

We evaluate PoVGEN via five research questions (RQs):

RQ1. How effective is PoVGEN?

RQ2. How does PoVGEN compare to other techniques?

RQ3. How does each of design components of PoVGEN contribute to its overall performance?

RQ4. How efficient is PoVGEN?

RQ5. Can PoVGEN reproduce real-world CVEs?

A. Datasets

We use three datasets for fine-tuning and evaluation.

Localization Fine-Tuning Dataset. For MLM fine-tuning, we adopt the InterPVD dataset [29], which provides labeled patches and manifestation points for 769 C/C++ vulnerabilities from 53 projects, covering 16 major CWEs.

Neural-Symbolic Fine-Tuning Dataset. For CEM and CSM fine-tuning, we collect 484 and 15,500 memory-safety samples from SV-COMP [38] and Juliet Test Suite [39],

TABLE III
EFFECTIVENESS OF PoVGEN COMPARED TO BASELINES ON THE ARVO DATASET.

Vulnerability Type	PoVGen	LibFuzzer	AFL++	AFLGo	KLEE
SEGV	72.73%	50.91%	47.27%	36.36%	3.64%
global-buffer-overflow	90.91%	50.00%	54.55%	45.45%	0.00%
heap-buffer-overflow	81.07%	49.79%	36.63%	45.27%	2.06%
heap-use-after-free	77.14%	57.14%	42.86%	54.29%	2.86%
stack-buffer-overflow	80.65%	35.48%	54.84%	48.39%	0.00%
stack-buffer-underflow	100.00%	33.33%	0.00%	33.33%	0.00%
stack-use-after-return	66.67%	66.67%	33.33%	100.00%	0.00%
undefined-behavior	56.25%	37.50%	50.00%	31.25%	6.25%
use-after-poison	85.71%	28.57%	71.43%	71.43%	0.00%
use-of-uninitialized-value	85.33%	58.67%	34.67%	40.00%	4.00%
<i>Overall</i>	80.20%	<i>50.20%</i>	<i>40.61%</i>	<i>44.49%</i>	<i>2.45%</i>

respectively. We further augment it with 10,000 synthetic samples generated from code templates, resulting in a combined set of 25,984 samples for neural symbolic fine-tuning.

Evaluation Datasets. For RQ1–RQ4, we use ARVO [10], which contains 5,000 memory vulnerabilities with ground-truth PoVs across 250+ C/C++ projects. We randomly sample 490 cases (98% confidence, 5% margin) spanning 110 projects and 10 vulnerability types (Table I).

To answer RQ5, we queried the CVE database for entries from 2012 to 2024. We filtered for vulnerabilities in C/C++ projects that had available patches but no listed PoVs. To streamline the setup of our evaluation environment, we focused on projects that are also in the ARVO dataset, which provides pre-built environments. This process resulted in 269 vulnerabilities, across 62 projects and 14 CWEs shown in Table II. We have checked to ensure that there is no overlap between the fine-tuning and evaluation datasets.

We note that our work focuses solely on generating PoVs – the concrete inputs that trigger the vulnerabilities in executable programs. Environment setup and harness building are outside our scope and have been addressed by prior work, with ARVO [10] providing pre-built projects and harnesses that we leverage for scalable evaluation.

Besides, the scope of our evaluation is focused on vulnerabilities in C/C++ projects. This decision is motivated by the prevalence of memory-safety vulnerabilities within this ecosystem, the availability of high-quality datasets like ARVO [10], and the significant impact of generating concrete PoVs for these types of bugs. However, we argue that PoVGEN’s modular design allows for extension to other vulnerability types and programming languages, which will also be discussed in §VI-B.

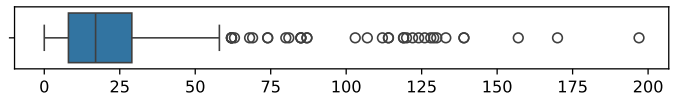


Fig. 5. Distribution of the number of paths needed to generate success PoVs.

B. RQ1: Effectiveness

To evaluate PoVGEN, we first generate a concrete PoV input, either by running the Python script outputted from the constraints solving model or by using the output from the SMT solver. Then, we run the sample’s harness with the generated PoV as input. A reproduction is considered successful if the program’s sanitizer reports an error where

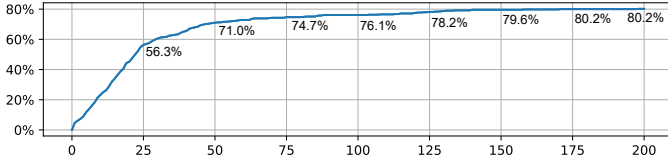


Fig. 6. Success rate of PoVGEN in terms of number of paths explored.

TABLE IV
EFFECTIVENESS OF PoVGEN WITHOUT PATCH INFORMATION

Vulnerability Type	Succ%	Vulnerability Type	Succ%
SEGV	79.91%	stack-buffer-underflow	33.33%
global-buffer-overflow	63.64%	stack-use-after-return	66.67%
heap-buffer-overflow	71.60%	undefined-behavior	43.75%
heap-use-after-free	65.71%	use-after-poison	71.43%
stack-buffer-overflow	74.19%	use-of-uninitialized-value	74.67%
Overall	70.20%		

both the vulnerability type and the code location of the crash match those triggered by the ground-truth PoV.

Table III illustrates the effectiveness of PoVGEN across different types of vulnerabilities in the ARVO dataset under the patch-guided mode. PoVGEN achieves an overall success rate of 80.20% in reproducing the vulnerabilities. The success rates across vulnerability types show that PoVGEN performs consistently well. It achieves particularly high accuracy on memory-related bugs (e.g., 90.91% for global-buffer-overflow and 100% for stack-buffer-underflow) and strong results on other cases. Overall, the results indicate that PoVGEN maintains stable effectiveness across diverse vulnerabilities.

We further examine how many execution paths PoVGEN needs. The path-count distribution is heavily skewed toward low numbers (Figure 5): the median is approximately 20 paths, and 75% of successful reproductions occur within the first 30 paths. The cumulative success rate (Figure 6) reaches 71.0% within 50 paths and then plateaus, indicating that PoVGEN’s guided exploration converges quickly.

To evaluate patch-free mode, we remove patch information from ARVO samples to simulate zero-day scenarios. Table IV shows PoVGEN reproduces 70.20% of vulnerabilities, indicating that the framework remains effective on unpatched code. The drop from patch-guided mode is driven by Step 1: without the patch signal, manifestation localization recall falls from 80.74% to 63.81% (Table V). The remaining stages still generate correct PoVs from this less precise starting point, showing their robustness.

PoVGEN achieves an 80.20% success rate in patch-guided mode and 70.20% in patch-free mode, with most successful PoVs generated within 50 execution paths.

C. RQ2: Comparison to Existing Techniques

Comparison to Fuzzers. We compare PoVGEN against state-of-the-art fuzzers: LibFuzzer [40], AFL++ [41], and the directed fuzzer AFLGo [42]. We use each ARVO project’s official seed corpus and, for AFLGo, configure the patch locations as targets to give it contextual information comparable to PoVGEN’s. To account for the randomness of fuzzing, each

TABLE V
MANIFESTATION LOCALIZATION EFFECTIVENESS

Approach	Recall	Precision	F1
PoVGEN	80.74%	58.48%	67.83%
PoVGEN-NOPATCH	63.81%	40.08%	49.23%
VulTrigger	<u>30.63%</u>	<u>14.67%</u>	<u>19.83%</u>
Gemini-2.5-Flash	55.25%	43.79%	48.86%
Llama-3.2	32.68%	22.59%	26.71%

TABLE VI
EFFECTIVENESS OF PoVGEN COMPARED TO STATE-OF-THE-ART LLMs ON THE PARTIAL ARVO DATASET.

Vulnerability Type	PoVGen	Gemini-2.5	OpenAI-o3	Claude-4
SEGV	83.33%	50.00%	33.33%	16.67%
global-buffer-overflow	100.00%	25.00%	25.00%	25.00%
heap-buffer-overflow	78.57%	21.43%	35.71%	17.86%
heap-use-after-free	66.67%	66.67%	66.67%	66.67%
use-of-uninitialized-value	77.78%	55.56%	33.33%	22.22%
Overall	79.91%	34.83%	35.89%	22.01%

experiment was *repeated three times*, and the reported success rate represents the *union* of vulnerabilities reproduced across all runs. The timeout is one week per sample. As shown in Table III, although the ARVO vulnerabilities were originally found by OSS-Fuzz, the fuzzing baselines achieved limited success within the one-week timeout, indicating that PoVGEN is more effective at reproducing known vulnerabilities.

Comparison to Symbolic Execution Engine. We also compare PoVGEN against KLEE [16]. We use each sample’s LLVM IR bitcode, modify the fuzz harness to mark inputs as symbolic, write stubs for sanitizer functions, and enable all of KLEE’s memory safety checkers. To account for its non-deterministic search, we repeated each run three times with a one-week timeout.

Table III shows that KLEE reproduces only 2.45% of the vulnerabilities. This reflects several well-known limitations of symbolic execution on real-world code: path explosion prevents enumerating enough paths within the budget; uClibc and manual modeling leave gaps for external and system calls, and real-world libraries (e.g., the image and audio parsers in ARVO) hit unimplemented externals that abort exploration; and KLEE’s flat byte-level memory model struggles with symbolic pointer arithmetic, complex aliasing, and large symbolic allocations common in memory-safety vulnerabilities.

While PoVGEN’s primary configuration uses patch information to guide its search and the fuzzing/KLEE baselines do not, the margin by which PoVGEN outperforms them already highlights the effectiveness of its methodology. PoVGEN also works without patches and still outperforms them (§VI-B).

Comparison to LLM-based Approaches. Recently, LLMs have been explored for vulnerability analysis. We thus compare PoVGEN with LLM-based PoV generation approaches. We use the most powerful commercial LLMs: OpenAI-o3, Claude-4-Sonnet, and Gemini-2.5-Pro. To conduct a fair comparison to PoVGEN that mirrors an expert’s workflow, we provide each model the official CVE description, the patch in diff format, and the source code of the program.

To guide the model’s reasoning, we employ a multi-step

TABLE VII
CEM CONSTRAINT EXTRACTION ACCURACY ON THE 52-SAMPLE SUBSET.

Approach	Precision		Recall	
	Syntax	Semantics	Syntax	Semantics
POVGEN CEM (fine-tuned)	69.38%	61.52%	94.81%	89.63%
Llama-3.1-8B (base)	20.11%	18.64%	60.74%	57.04%
Gemini-2.5-Flash	60.42%	51.14%	87.32%	83.10%
Claude-4-Sonnet	68.72%	59.24%	92.52%	86.14%

TABLE VIII
CSM PoV GENERATION SUCCESS RATE ON THE 52-SAMPLE SUBSET,
GIVEN THE SAME CEM-EXTRACTED CONSTRAINTS.

Approach	Success Rate
POVGEN CSM (fine-tuned)	76.51%
Llama-3.1-8B (base)	37.96%
Gemini-2.5-Flash	67.96%
Claude-4-Sonnet	73.46%
Z3 only	51.22%

prompting strategy. The prompt instructs the LLM to be an expert cybersecurity researcher and to deconstruct the problem by first identifying the vulnerability manifestation point, then formulating a viable execution path to that point, next deriving the necessary input constraints for that path, and finally generating a PoV script. The full prompt template detailing this structured task is in Appendix B2.

Due to the context window limitations of these commercial LLMs (e.g., OpenAI-o3 and Claude-4-Sonnet allow up to 200k tokens), we were only able to select 52 samples that fit. These samples include 6 SEGV, 4 global-buffer-overflow, 28 heap-buffer-overflow, 3 heap-use-after-free, and 11 use-of-uninitialized-value. Considering the cost of these LLMs, we limited the models to a maximum of 10 trials if the generated PoVs failed to trigger the vulnerability.

As shown in Table VI, even when guided through a POVGEN-like workflow, the LLM baselines underperform significantly. Thus, POVGEN far outperforms pure LLM-based attempts—79.91% vs 35.89% (OpenAI-o3)—underscoring the necessity of its specialized, fine-tuned pipeline over high-level reasoning alone.

POVGEN is far more effective than traditional fuzzing (max 50.20%), symbolic execution (2.45%) and LLM-based approaches (max 35.89%).

D. RQ3: Ablation Studies

To answer RQ3, we conduct ablation studies to evaluate the contribution of each key component.

We first investigate the impact of different processing steps for neural-symbolic fine-tuning and PoV generation. As shown in Table IX “No SMT Solver”, removing the SMT solver from PoV generation results in a drop in the success rate from 80.20% to 77.76%, suggesting the benefit of integrating a traditional SMT solver for PoV generation. To isolate the effects of other components, the SMT solver is disabled in all the following ablation experiments. When we remove the constraint simplification/optimization, as shown in “No Simp”, the success rate drops further to 76.12%. When removing both constraint simplification and augmentation, denoted as “No Simp&Aug”, the success rate falls to 74.90%. This decline

highlights that both the augmentation fine-tuning data with templates and the subsequent constraints optimization are effective for PoV generation.

The flow-based augmentation, constraints optimization, and the integration of SMT solver for neural-symbolic fine-tuning and PoV generation are all crucial for the effectiveness of POVGEN.

Next, we evaluate the impact of our fine-tuned neural-symbolic PoV generation models in Step 3 by replacing them with their base model and a powerful commercial LLM. In the “Non-Fine-Tune/Powerful S3” and “No-Fine-Tune/Base S3” experiments, we substitute our fine-tuned models for constraints extraction and constraints solving with the Gemini-2.5-Flash and Llama-3.1-8b models, respectively. We chose Gemini-2.5-Flash over more powerful LLMs in RQ2 like Gemini-2.5-Pro due to the extensive token requirements of our approach; on average, each sample consumes 4.57 million tokens, which would cost \$1.37 per sample on the more economical Gemini-2.5-Flash (at \$0.30 per million tokens). The results in Table IX show a substantial performance decrease, with the success rate dropping to 60.41% for Gemini and 25.31% for Llama3. This demonstrates the benefit of fine-tuning LLMs on our specific data for neural symbolic execution.

Fine-tuning large language models on our specific data for neural symbolic execution is critical for achieving high success rates in PoV generation.

We then directly measure the accuracy of the fine-tuned CEM and CSM, which complements the end-to-end ablations above and characterizes how fine-tuning helps each model. We use the 52-sample subset from RQ2 (the samples that fit commercial LLM context windows) and evaluate the top 50 paths per sample, since Figure 5 shows that 50 paths cover the majority of successful PoVs. For CEM, two authors independently labeled each generated constraint set on two criteria: *syntax validity* (whether the output is a well-formed SMT-LIB formula) and *semantics correctness* (whether the constraints capture the main vulnerability-triggering condition well enough to support PoV generation). Both annotators were blinded to which model produced each output, and inter-annotator agreement was Cohen’s $\kappa = 0.81$. We report *precision* (fraction of all generated constraint sets that are correct) and *recall* (fraction of samples for which at least one of the 50 paths produces a correct constraint set). For CSM, we feed the same set of CEM-extracted constraints to each candidate solver and report the success rate (the fraction of samples for which a generated PoV triggers the target vulnerability).

Table VII reports CEM accuracy across POVGEN’s fine-tuned CEM, the base Llama-3.1-8B, and two frontier commercial LLMs. The fine-tuned CEM achieves the highest recall on both syntax (94.81%) and semantics (89.63%), the metric that matters most for end-to-end PoV success since only one

TABLE IX
HOW THE POVGEN COMPONENTS CONTRIBUTE TO THE PERFORMANCE

Vulnerability Type	PoVGen	Ablated Version of PoVGen								
		No SMT Solver	No Simp	No Simp&Aug	Non-Fine-Tune/Powerful S3	No-Fine-Tune/Base S3	No S1	VulTrigger S1	Non-Fine-Tune/Powerful S1	No-Fine-Tune/Base S1
SEGV	72.73%	67.27%	74.55%	74.55%	41.82%	25.45%	63.64%	62.50%	61.54%	62.50%
global-buffer-overflow	80.91%	86.36%	81.82%	86.36%	50.00%	18.18%	63.64%	50.00%	75.00%	71.43%
heap-buffer-overflow	81.07%	79.01%	75.72%	74.49%	63.37%	24.28%	62.14%	68.97%	71.88%	66.94%
heap-use-after-free	77.14%	77.14%	80.00%	68.57%	74.29%	37.14%	74.29%	58.33%	75.00%	74.07%
stack-buffer-overflow	80.65%	77.42%	74.19%	74.19%	58.06%	12.90%	70.97%	44.44%	68.75%	50.00%
stack-buffer-underflow	100.00%	100.00%	66.67%	66.67%	0.00%	0.00%	66.67%	50.00%	50.00%	0.00%
stack-use-after-return	66.67%	66.67%	33.33%	66.67%	33.33%	0.00%	66.67%	100.00%	0.00%	100.00%
undefined-behavior	56.25%	56.25%	50.00%	56.25%	31.25%	6.25%	43.75%	100.00%	28.57%	60.00%
use-after-poison	85.71%	71.43%	71.43%	71.43%	57.14%	28.57%	42.86%	100.00%	33.33%	100.00%
use-of-uninitialized-value	85.33%	84.00%	84.00%	81.33%	72.00%	36.00%	70.67%	65.52%	67.35%	66.67%
All	80.20%	77.76%	76.12%	74.90%	60.41%	25.31%	64.29%	65.38%	68.32%	66.54%

correct constraint set per sample is needed to generate a PoV. The base Llama is far behind on every metric, confirming that the gain comes from fine-tuning rather than from the underlying model. The fine-tuned CEM matches or slightly exceeds Claude-4-Sonnet (89.63% vs. 86.14% semantic recall), which is notable given the cost difference: Claude-4-Sonnet costs \$2.06 per sample (Table X), while the fine-tuned CEM has zero marginal cost. At 52 samples, that is a \$107 difference in API spend for comparable accuracy.

Table VIII shows that the fine-tuned CSM achieves the highest success rate (76.51%), again at zero marginal cost. Compared to Claude-4-Sonnet at 73.46%, the gap is modest, but at \$2.06 per sample Claude-4-Sonnet costs roughly \$2.81 per successful PoV across the 52 samples while CSM costs \$0. The Z3 *only* row feeds the same CEM-extracted constraints directly to the SMT solver without CSM, and it achieves 51.22%, well below CSM. This isolates CSM’s contribution beyond solver fallback, and also explains the gap between Z3 here (51.22%) and KLEE in Table III (2.45%): both use Z3, but CEM provides path-focused constraints that the solver can handle, whereas KLEE’s whole-program symbolic execution suffers from path explosion and external-call modeling gaps.

The fine-tuned CEM and CSM match or exceed frontier commercial LLMs on direct accuracy at zero marginal cost. The CEM + Z3 baseline (51.22%) shows that CSM contributes value beyond solver fallback and that CEM produces tractable constraints that whole-program symbolic execution cannot.

Finally, we assess the impact of Vulnerability Manifestation Localization (Step 1). In the “No S1” experiment, removing Step 1 entirely and relying only on execution paths to the patch location causes the success rate to drop from 80.20% to 64.29%. This marked decrease confirms that vulnerability manifestation localization is crucial for effective PoV reproduction.

We also tested alternative manifestation localization approaches, including VulTrigger and general-purpose LLMs (Gemini-2.5-Flash, Llama-3.2-3B). Table V shows that these baselines struggle compared to POVGEN’s fine-tuned model, which achieves 80.74% recall and 67.83% F1. The “Non-Fine-Tune/Powerful S1” and “No-Fine-Tune/Base S1” rows in Table IX show a clear correlation between localization recall and final PoV success.

Notably, POVGEN’s end-to-end success rate (80.20%) nearly matches its Step 1 recall (80.74%), indicating that Steps

TABLE X
AVERAGE TIME AND LLM COST PER SAMPLE FOR POVGEN AND BASELINES

Approach	Time (min)	API Cost (\$)
POVGEN	98.13	0.02
—Step 1	0.53	0.02
—Step 2	75.71	0
—Step 3	21.89	0
libFuzzer	260.49	0
AFL++	729.38	0
AFLGo	625.17	0
KLEE	1122.23	0
Gemini-2.5-Pro	8.23	1.72
OpenAI-o3	5.64	1.37
Claude-4	4.32	2.06

2 and 3 are robust: as long as the predicted location falls on the correct control-flow path, they can derive the necessary constraints even when the exact trigger statement is missed. Step 1’s high recall is still essential to provide that correct path.

Step 1 is crucial, as removing it reduces the success rate. The fine-tuned manifestation localization model outperforms other techniques helping POVGEN achieve higher success rates.

E. RQ4: Efficiency

Table X presents the efficiency of POVGEN and the baselines in terms of time and monetary cost.

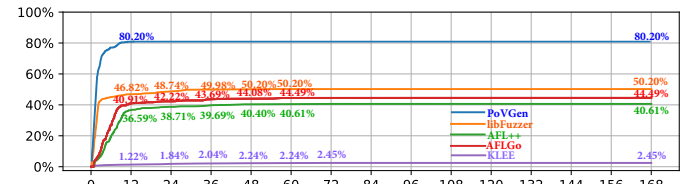


Fig. 7. Progression of success rate (y axis) achieved at increasing time cost in terms of hours spent (x axis).

POVGEN takes an average of 98.13 minutes to reproduce a vulnerability. The majority of this time is spent in Step 2 (Path-Sensitive Reachability Exploration, 75.71 minutes) and Step 3 (Neural-Symbolic PoV Generation, 21.89 minutes), which is expected as these steps involve exploring a large number of execution paths and generating constraints. In terms of monetary cost, only Step 1 incurs a cost, averaging \$0.02 per sample for commercial LLM (Gemini-2.5-Flash) usage.

In comparison, fuzzing takes a much longer time. Figure 7 shows that the fuzzers (libFuzzer, AFL++, and AFLGo) struggle to reproduce more vulnerabilities after 48 hours. Even

for the successful cases, fuzzing takes an average of 260.49 minutes per sample for LibFuzzer, and 729.38 and 625.17 minutes with AFL++ and AFLGo, respectively (Table X), much less efficient than POVGEN for this task. The symbolic execution engine KLEE is the least efficient, requiring an average of 1122.23 minutes for the few vulnerabilities it successfully reproduced.

POVGEN is significantly more efficient than fuzzing and symbolic execution in terms of time and substantially more cost-effective than powerful commercial LLMs.

F. RQ5: Real-World CVE Reproduction

TABLE XI

EFFECTIVENESS OF POVGEN AGAINST THE REAL-WORLD CVE DATABASE

CWE ID	%Success	CWE ID	%Success	CWE ID	%Success
CWE-119	68.18%	CWE-190	100.00%	CWE-415	77.78%
CWE-120	88.89%	CWE-193	100.00%	CWE-416	76.00%
CWE-122	100.00%	CWE-20	81.82%	CWE-476	70.37%
CWE-125	79.22%	CWE-399	100.00%	CWE-787	67.35%
CWE-1284	100.00%	CWE-401	100.00%	All	75.09%

Table XI presents the effectiveness of POVGEN on the real-world CVE dataset, where the PoVs are not available. Overall, POVGEN successfully generated a valid PoV for 75.09% of the 269 vulnerabilities, demonstrating its strong capability in a practical, real-world setting. The performance is notably strong for several vulnerability classes, achieving 100.00% success rates for CWE-122, CWE-1284, CWE-190, CWE-193, CWE-399, and CWE-401. While generally effective, the success rates vary across different CWEs. For instance, CWE-125 (Out-of-bounds Read) shows a 79.22% success rate, and CWE-415 (Double Free) shows a success rate of 77.78%. These differences reflect the varying complexity of CWEs, but the overall trend remains stable.

POVGEN is highly effective at automatically generating PoVs for real-world, disclosed CVEs, even when no public PoVs are available.

VI. DISCUSSION

We further discuss the advantages of POVGEN, its current limitations, and its extensibility.

A. Why POVGEN Works

POVGEN’s effectiveness stems from its systematic design. First, POVGEN narrows the search space by anchoring exploration to the manifestation point (Challenge 1). As shown in Figure 8, manifestation localization correctly pinpoints line 1760 even when the patch is at line 2360, and the guided random walk uses this anchor to extract a single focused trace (Figure 9) from a project containing thousands of functions.

Second, POVGEN handles constraint identification and solving on this focused trace by combining symbolic tools with fine-tuned LLMs (Challenge 2). As shown in Figure 10, the fine-tuned CEM extracts SMT constraints from the trace, and the fine-tuned CSM synthesizes a working PoV from those constraints, with a traditional SMT solver as fallback.

Third, fine-tuning two specialized LLMs on traces from traditional symbolic tools closes the distributional gap between the formal sub-tasks and frontier pretraining (Challenge 3).

TABLE XII
FLAWED PATCHES FOUND BY POVGEN

CVE	Project	CWE	Patch Issue
CVE-2020-15471	nDPI	CWE-125	Incomplete
CVE-2020-15473	nDPI	CWE-125	Incorrect
CVE-2020-15475	nDPI	CWE-416	Incomplete
CVE-2020-26570	OpenSC	CWE-787	Incorrect
CVE-2022-23537	PJSIP	CWE-125	Incomplete
CVE-2023-27599	OpenSIPS	CWE-20	Incomplete

The fine-tuned models extract and solve SMT constraints from focused execution paths and synthesize working PoVs (Figure 10), while running locally amortizes their one-time training cost into zero marginal cost per inference.

In contrast, naive LLM application is ineffective. As discussed in §V-C, prompting a powerful model like Gemini-2.5-Pro with an entire project fails because the model has no mechanism to narrow the search space to the true manifestation point (Challenge 1), producing a generic and incorrect PoV (Figure 12). Even when provided with a focused execution path, a general-purpose LLM still struggles: as shown in §V-D and Figure 13, a non-fine-tuned model lacks the specialized capability for formal constraint extraction (Challenge 2), producing incorrect SMT2 constraints and a flawed PoV script. This shows that narrowing the context alone is insufficient without fine-tuning (Challenge 3).

B. Applicability Scope

Applicability to Other Types of Vulnerabilities and Languages. POVGEN’s modular design facilitates extension beyond C/C++ memory vulnerabilities. Other vulnerability classes like SQL injection require integrating corresponding oracle tools (e.g., sanitizers) to validate PoVs, and other languages like Java or Python require replacing language-specific components such as SVF.

Applicability of Individual Components. POVGEN’s components also work as standalone tools: CEM and CSM form an LLM-based symbolic execution engine that processes execution traces from any path-discovery tool, and MLM pinpoints vulnerability locations with or without patch information.

TABLE XIII
ZERO-DAY VULNERABILITIES FOUND BY POVGEN

Project	Version	Vulnerability Type
libucl	0.9.1	Heap-buffer-overflow
libucl	0.9.1	Memory leak
radare2	5.9.2	Memory leak
matio	1.5.27	Memory leak
gstreamer	1.24.3	Stack-buffer-overflow

C. Usefulness of PoVs

The successful generation of PoVs is not only crucial for reproducing known vulnerabilities but is also useful for validating patches. To demonstrate this, on the CVE dataset, we conduct an experiment that takes the PoVs generated for a given vulnerability and executes them against the patched version. As shown in Table XII, POVGEN identifies six flawed patches in the CVE dataset. Among them, four patches are incomplete, where the PoVs still trigger the original crashes, and two are incorrect, where the PoVs trigger different crashes. This demonstrates the practical utility of POVGEN as a tool for patch validation.

Beyond validating patches for known vulnerabilities, the PoVs generated by POVGEN can also be used for discovering new, zero-day vulnerabilities. To demonstrate this, we apply the PoVs generated from POVGEN for the ARVO dataset to more recent versions of the same projects to check for the new vulnerabilities. As shown in Table XIII, we identified and reported five previously unknown zero-day vulnerabilities. This highlights the potential of using POVGEN’s generated PoVs to identify zero-day vulnerabilities on the latest software versions.

The generated PoVs also serve broader cybersecurity needs, providing high-quality ground-truth data for training analysis tools and offering concrete examples for security education. Critically, POVGEN links each successful PoV to its triggering execution path. This can be useful for developers to debug, analyze root causes, and develop fixes.

D. Comparison with LLM-Driven Concolic Execution

Recent work uses LLMs to drive concolic execution, which operates in a different setting from POVGEN. ConcoLLMic [24] uses LLM agents to symbolize inputs and summarize constraints, and CottonTail [25] uses an LLM-driven solver with a Solve-Complete paradigm to generate syntactically valid test inputs. Both are dynamic and require concrete seed inputs to drive symbolic exploration along observed runs. CottonTail can also generate seeds via LLM, but its exploration goal is parser coverage rather than reaching a specific disclosed vulnerability. POVGEN’s primary use case is the opposite—reproducing CVEs that lack public PoVs—so it performs a static neural-symbolic process, generating inputs from scratch by exploring viable paths and extracting constraints without concrete executions. The approaches are complementary: concolic agents are useful when seed inputs are available or coverage is the goal, while POVGEN is designed to reproduce specific vulnerabilities for which neither seeds nor public PoVs exist.

A second difference is design philosophy. ConcoLLMic and CottonTail wrap general-purpose commercial LLMs (e.g., GPT-4o, GPT-4o-mini) in scaffolding (planning, retrieval, tool use), which is effective for open-ended exploration but incurs per-call API cost—CottonTail reports approximately \$0.78/hour [25]—and depends on commercial models that change over time. POVGEN fine-tunes local models for the narrow formal sub-tasks of constraint extraction and solving, trading flexibility for output regularity, zero marginal cost, and reproducibility. RQ3 (§V-D) shows that for these sub-tasks, the fine-tuned models match or exceed frontier commercial LLMs.

E. Limitation

We discuss four limitations. First, Step 3’s fine-tuning data quality is bounded by the simplicity of standard benchmarks (SV-COMP, Juliet); we mitigate this with four augmentation templates and an SMT fallback when the solving model fails. Second, Step 2’s whole-program analysis can be expensive; we use SVF’s lightweight control-flow module rather than its data-flow module, and pre-compute distance maps with parallel processing to mitigate path explosion. Third, POVGEN

requires a compilable program with defined entry points and does not configure build systems or generate harnesses; instead, it integrates with frameworks like ARVO that provide pre-built environments. Fourth, POVGEN’s fine-tuned models handle integer and pointer constraints well but may struggle with floating-point or non-linear operations; the SMT fallback serves as a deterministic check, applied only to the focused paths from Steps 1-2 to avoid the scalability bottlenecks of whole-program symbolic execution.

VII. RELATED WORK

Symbolic execution and fuzzing. Early Automatic Exploit Generation (AEG) utilized symbolic execution with SMT solvers, as seen in KLEE [16] and SAGE [43]. To overcome the limitations of random testing from fuzzers like AFL [44], hybrid techniques such as Driller [15] and AFL++ [41] emerged, integrating symbolic execution to reach deeper code. Patch-based AEG also uses symbolic analysis on binary diffs [45]. However, these AEG techniques rely on heavyweight symbolic execution engines that strictly track all path constraints, making them computationally infeasible for large, real-world codebases due to path explosion. In contrast, POVGEN overcomes this by employing a lightweight guided search to isolate single viable traces before constraint solving.

Vulnerability-specific PoV/exploit generators. Many systems are tailored to narrow vulnerability classes. Examples include tools for PHP object injection (Fugio [18]), Node.js package exploits (Explode.js [19]), dynamic web applications (NAVEX [21]), and heap overflows in kernels (KOOBE [20]) or language interpreters (Gollum [46], AAHEG [47]). While effective within their domains, these tools are tightly coupled to specific language semantics or execution environments, requiring significant re-engineering to adapt to new contexts. In contrast, POVGEN abstracts the problem into logical constraint satisfaction, which decouples the exploit generation logic from language-specific exploitation heuristics.

Machine learning and LLM-based approaches. Learning-based methods have gained traction, starting with imitation learning for fuzzing smart contracts [48]. Recently, the field has witnessed the emergence of agentic symbolic execution, where LLMs act as intelligent agents to augment traditional analysis. For instance, ConcoLLMic leverages LLM agents to drive concolic execution by symbolizing inputs and summarizing constraints [24]. LangSym generates test cases to guide the symbolic search space for higher coverage [49]. Hybrid approaches such as AutoBug use LLMs as path constraint solvers [23], and SymGPT translates natural-language specifications for verifying smart contracts [50]. These agentic techniques improve scalability and code coverage, but primarily focus on bug detection or verification rather than generating concrete PoVs. Concolic approaches in particular are dynamic and require concrete seed inputs to drive exploration, which are unavailable for CVEs without public PoVs. POVGEN instead operates statically, generating inputs from scratch through guided path exploration and constraint extraction.

Other works apply LLMs to generate PoVs (PoCGen [22]), symbolically trace vulnerabilities (FaultLine [51]), solve CTF challenges (PwnGPT [52]), and penetration testing (Pentest-GPT [53]). Though promising, direct LLM prompting lacks the rigorous formal reasoning required to calculate precise byte-level constraints (e.g., integer overflows or pointer arithmetic), resulting in plausible-looking but functionally invalid PoVs. In comparison, POVGEN fine-tunes models specifically to translate code semantics into SMT constraints, bridging the gap between high-level language understanding and low-level constraint solving without depending on concrete seed inputs or commercial LLM APIs.

VIII. CONCLUSION

We introduce POVGEN, a framework that combines vulnerability localization, path-sensitive exploration, and fine-tuned neural-symbolic generation to automatically produce PoVs. POVGEN achieves an 80.20% success rate on benchmarks and 75.09% on real-world CVEs lacking PoVs, significantly outperforming fuzzing and LLM prompting. Furthermore, POVGEN identified five zero-day vulnerabilities and six flawed patches, proving its practical utility for vulnerability reproduction. All artifacts, including documentation of the identified flawed patches and zero-day vulnerabilities, are available at <https://figshare.com/s/6393c45416d0fcf2d77a>.

ETHICS CONSIDERATIONS

This work generates Proof-of-Vulnerability (PoV) inputs for software vulnerabilities, which carries risks: PoVs that confirm a vulnerability for defenders can also serve as exploitation primitives for attackers. We considered this risk throughout the research and took the following steps to mitigate it.

Scope and target selection. Our evaluation focuses on disclosed CVEs from the ARVO and CVE databases that already have public patches. For these vulnerabilities, defenders already have remediation available, so PoV generation helps with patch verification rather than enabling new attacks. We did not generate PoVs for undisclosed vulnerabilities outside the zero-day discovery process described below.

Responsible disclosure of zero-day vulnerabilities. POVGEN identified five previously unknown vulnerabilities in libucl, radare2, matio, and gstreamer (Table XIII). We followed responsible disclosure: each vulnerability was reported to the maintainers through their security channels before submission of this paper, with details for the maintainers to reproduce and fix the issue. We did not publish PoV inputs or exploitation details ahead of patches being available.

Flawed patch findings. POVGEN identified six flawed patches for previously disclosed CVEs (Table XII). Because the underlying vulnerabilities are already public, the additional disclosure risk from reporting that a patch is incomplete or incorrect is bounded. We notified the affected projects (nDPI, OpenSC, PJSIP, OpenSIPS) of the flawed patches through their security channels.

Artifact release. The released artifact contains the POVGEN framework, fine-tuning data derived from public benchmarks (SV-COMP, Juliet), and the evaluation pipeline. It does not include PoV inputs for the zero-days reported above until the corresponding patches have been released by upstream maintainers.

No human subjects or sensitive data. The research did not involve human subjects, user data, or any data subject to privacy regulation. All evaluation data is drawn from publicly available CVE records and open-source software.

We believe these measures bring the research into alignment with the Menlo Report principles of beneficence, respect for persons, justice, and respect for law and public interest, and that the defensive value of automated PoV generation for patch verification and zero-day discovery outweighs the residual dual-use risk.

REFERENCES

- [1] Ericsson, “Software vulnerability: Impact & ways to avoid it,” <https://www.ericsson.com/en/security/vulnerability-management>, 2023.
- [2] NIST, “National vulnerability database (NVD) dashboard,” <https://nvd.nist.gov/general/nvd-dashboard>, 2023.
- [3] X. Fu and H. Cai, “FlowDist: Multi-staged refinement-based dynamic information flow analysis for distributed software systems,” in *USENIX Security*, 2021.
- [4] W. Li, J. Ming, X. Luo, and H. Cai, “PolyCruise: A cross-language dynamic information flow analysis,” in *USENIX Security*, 2022.
- [5] W. Li, J. Ruan, G. Yi, L. Cheng, X. Luo, and H. Cai, “PolyFuzz: Holistic greybox fuzzing of multi-language systems,” in *USENIX Security*, 2023.

- [6] W. Li, H. Yang, X. Luo, L. Cheng, and H. Cai, "PyRTFuzz: Detecting bugs in python runtimes via two-level collaborative fuzzing," in *CCS*, 2023.
- [7] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, "Examining zero-shot vulnerability repair with large language models," in *S&P*, 2023.
- [8] X. Zhou, S. Cao, X. Sun, and D. Lo, "Large language model for vulnerability detection and repair: Literature review and roadmap," *arXiv preprint arXiv:2404.02525*, 2024.
- [9] Y. Wu, N. Jiang, H. V. Pham, T. Lutellier, J. Davis, L. Tan, P. Babkin, and S. Shah, "How effective are neural networks for fixing security vulnerabilities," in *ISSSTA*, 2023.
- [10] X. Mei, P. S. Singaria, J. Del Castillo, H. Xi, T. Bao, R. Wang, Y. Shoshitaishvili, A. Doupé, H. Pearce, B. Dolan-Gavitt *et al.*, "ARVO: Atlas of reproducible vulnerabilities for open source software," *arXiv preprint arXiv:2408.02153*, 2024.
- [11] F. Li and V. Paxson, "A large-scale empirical study of security patches," in *CCS*, 2017.
- [12] H. Wu, J. Wu, R. Wu, A. Sharma, A. Machiry, and A. Bianchi, "Veribin: Adaptive verification of patches at the binary level," in *NDSS*, 2025.
- [13] Jason Kikta, "The shrinking window for vulnerability exploitation: Why automated patching is critical," <https://www.automox.com/blog/why-automated-patching-is-critical>, 2025.
- [14] K. Serebryany, "OSS-Fuzz-google's continuous fuzzing service for open source software," 2017.
- [15] N. Stephens, J. Groten, C. Salls, A. Dutcher, R. Wang, J. Corbetta, Y. Shoshitaishvili, C. Kruegel, and G. Vigna, "Driller: Augmenting fuzzing through selective symbolic execution," in *NDSS*, 2017.
- [16] C. Cadar, D. Dunbar, D. R. Engler *et al.*, "KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs," in *OSDI*, 2008.
- [17] Y. Wang, Z. Li, and Y. Zhang, "PBAEG: combine-vulnerabilities aeg to defeat protection mechanisms," *Cybersecurity*, 2025.
- [18] S. Park, D. Kim, S. Jana, and S. Son, "FUGIO: Automatic exploit generation for PHP object injection vulnerabilities," in *USENIX Security*, 2022.
- [19] F. Marques, M. Ferreira, A. Nascimento, M. E. Coimbra, N. Santos, L. Jia, and J. Frago Santos, "Automated exploit generation for Node.js packages," *PLDI*, 2025.
- [20] W. Chen, X. Zou, G. Li, and Z. Qian, "KOOBE: Towards facilitating exploit generation of kernel out-of-bounds write vulnerabilities," in *USENIX security*, 2020.
- [21] A. Alhuzali, R. Gjomemo, B. Eshete, and V. Venkatakrishnan, "NAVEX: Precise and scalable exploit generation for dynamic web applications," in *USENIX Security*, 2018.
- [22] D. Simsek, A. Eghbali, and M. Pradel, "PoCGen: Generating proof-of-concept exploits for vulnerabilities in npm packages," *arXiv preprint arXiv:2506.04962*, 2025.
- [23] Y. Li, R. Meng, and G. J. Duck, "Large language model powered symbolic execution," *OOPSLA*, 2025.
- [24] Z. Luo, H. Zhao, D. Wolff, C. Cadar, and A. Roychoudhury, "Agentic concolic execution," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2026, pp. 1–19.
- [25] H. Tu, S. Lee, Y. Li, P. Chen, L. Jiang, and M. Böhme, "Cottontail: Large language model-driven concolic execution for highly structured test input generation," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, IEEE, 2026, pp. 1–19.
- [26] National Institute of Standards and Technology (NIST), "National Vulnerability Database (NVD)," <https://nvd.nist.gov>, 2025.
- [27] Y. Nong, H. Yang, L. Cheng, H. Hu, and H. Cai, "APPATCH: Automated adaptive prompting large language models for real-world software vulnerability patching," in *USENIX Security*, 2025.
- [28] Y. Nong, M. Aldeen, L. Cheng, H. Hu, F. Chen, and H. Cai, "Chain-of-thought prompting of large language models for discovering and fixing software vulnerabilities," *arXiv preprint arXiv:2402.17230*, 2024.
- [29] Z. Li, N. Wang, D. Zou, Y. Li, R. Zhang, S. Xu, C. Zhang, and H. Jin, "On the effectiveness of function-level vulnerability detectors for interprocedural vulnerabilities," in *ICSE*, 2024.
- [30] S. Horwitz, T. Reps, and D. Binkley, "Interprocedural slicing using dependence graphs," *TOPLAS*, 1990.
- [31] K. K. Shahandashti, M. M. Mohajer, A. B. Belle, S. Wang, and H. Hemmati, "Program slicing in the era of large language models," *arXiv preprint arXiv:2409.12369*, 2024.
- [32] F. Yamaguchi, "A platform for robust analysis of c/c++ code," <https://joern.readthedocs.io/en/latest/installation.html>, 2022.
- [33] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "SySeVR: A framework for using deep learning to detect software vulnerabilities," *TDSC*, 2021.
- [34] Tristan Ravitch, "A wrapper script to build whole-program llvm bitcode files," <https://github.com/travitch/whole-program-llvm>, 2021.
- [35] Y. Sui and J. Xue, "SVF: interprocedural static value-flow analysis in llvm," in *CC*, 2016.
- [36] L. De Moura and N. Bjørner, "Z3: An efficient smt solver," in *TACAS*, 2008.
- [37] D. Han and M. Han, "Unslloth," <https://github.com/unsllothai/unslloth>, 2024.
- [38] D. Beyer, "Software verification: 10th comparative evaluation (SV-COMP 2021)," in *TACAS*, 2021.
- [39] P. E. Black, *Juliet 1.3 test suite: Changes from 1.2*. US Department of Commerce, National Institute of Standards and Technology, 2018.
- [40] K. Serebryany, "Continuous fuzzing with libfuzzer and addresssanitizer," in *SecDev*, 2016.
- [41] A. Fioraldi, D. Maier, H. Eibfeldt, and M. Heuse, "AFL++: Combining incremental steps of fuzzing research," in *WOOT*, 2020.
- [42] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury, "Directed greybox fuzzing," in *CCS*, 2017.
- [43] P. Godefroid, M. Y. Levin, and D. Molnar, "SAGE: whitebox fuzzing for security testing," *Communication of ACM*, 2012.
- [44] M. Zalewski, "American fuzzy lop (AFL) fuzzer," <https://lcamtuf.coredump.cx/afl>, 2017.
- [45] D. Brumley, P. Poosankam, D. Song, and J. Zheng, "Automatic patch-based exploit generation is possible: Techniques and implications," in *S&P*, 2008.
- [46] S. Heelan, T. Melham, and D. Kroening, "Gollum: Modular and greybox exploit generation for heap overflows in interpreters," in *CCS*, 2019.
- [47] Y. Wang, Y. Zhang, and Z. Li, "AAHEG: Automatic advanced heap exploit generation based on abstract syntax tree," *Symmetry*, 2023.
- [48] J. He, M. Balunović, N. Ambroladze, P. Tsankov, and M. Vechev, "Learning to fuzz from symbolic execution with application to smart contracts," in *CCS*, 2019.
- [49] J. Xu, J. Xu, T. Chen, and X. Ma, "Symbolic execution with test cases generated by large language models," in *2024 IEEE 24th International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 2024, pp. 228–237.
- [50] S. Xia, M. He, S. Shao, T. Yu, Y. Zhang, and L. Song, "SymGPT: Auditing smart contracts via combining symbolic execution with large language models," *arXiv preprint arXiv:2502.07644*, 2025.
- [51] V. Nitin, B. Ray, and R. Z. Moghaddam, "Faultline: Automated proof-of-vulnerability generation using llm agents," *arXiv preprint arXiv:2507.15241*, 2025.
- [52] W. Peng, L. Ye, X. Du, H. Zhang, D. Zhan, Y. Zhang, Y. Guo, and C. Zhang, "PwnGPT: Automatic exploit generation based on large language models," in *ACL*, 2025.
- [53] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *USENIX Security*, 2024.
- [54] "Gpt-4.1," <https://en.wikipedia.org/wiki/GPT-4.1>, 2025.
- [55] "Gemini (language model)," [https://en.wikipedia.org/wiki/Gemini_\(language_model\)](https://en.wikipedia.org/wiki/Gemini_(language_model)), 2025.
- [56] "Claude (language model)," [https://en.wikipedia.org/wiki/Claude_\(language_model\)](https://en.wikipedia.org/wiki/Claude_(language_model)), 2025.
- [57] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. T. Lee, Y. Li, S. Lundberg *et al.*, "Sparks of artificial general intelligence: Early experiments with gpt-4," *arXiv preprint arXiv:2303.12712*, 2023.
- [58] OpenAI, "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774*, 2023.
- [59] Y. Nong, H. Yang, L. Cheng, H. Hu, and H. Cai, "Automated software vulnerability patching using large language models," *arXiv preprint arXiv:2408.13597*, 2024.
- [60] J. He, G. Sivanrupan, P. Tsankov, and M. Vechev, "Learning to explore paths for symbolic execution," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 2526–2540.

A. Background

1) *Large Language Models (LLMs)*: Large Language Models (LLMs) have demonstrated remarkable progress across natural language and code-related tasks. Models such as GPT, Gemini, and Claude have shown strong capabilities in code generation, program repair, and software analysis [54], [55], [56]. Their ability to understand program semantics and generate contextually relevant code has enabled their adoption in domains that traditionally required expert human reasoning [57], [58].

In the field of software security, researchers have started to leverage LLMs for vulnerability-related tasks. Recent work has shown promising results in automated vulnerability patching [59], penetration testing [53], and PoV generation [52]. They demonstrate that LLMs are capable of not only analyzing vulnerabilities but also synthesizing concrete PoVs. While the reliability of LLM-generated PoVs is still evolving, these works highlight the growing role of LLMs in automating security-critical workflows.

2) *Symbolic Execution*: Symbolic execution is a program analysis technique that systematically explores execution paths by treating program inputs as symbolic variables rather than concrete values [16]. Instead of executing a program on one specific input, a symbolic execution engine maintains a set of constraints describing the constraints under which a particular path in the program would be taken. By solving these constraints, the engine can generate inputs that force the program down targeted paths, including those that expose vulnerabilities.

In practice, symbolic execution engines typically follow a structured workflow consisting of several steps:

- 1) **Symbolic Input Initialization**: Program inputs are marked as symbolic variables rather than concrete values.
- 2) **Instruction Execution and State Update**: The engine executes each instruction symbolically, updating the program state with symbolic expressions.
- 3) **Path Exploration**: When encountering a branch, the engine forks the execution state and updates the path constraints for each branch.
- 4) **Constraints Solving**: For each path, the accumulated path constraints is submitted to an SMT solver. If satisfiable, the solver provides a concrete input assignment.

This methodology can be used in vulnerability discovery and PoV generation. For example, KLEE has been widely adopted for generating high-coverage test cases for complex systems software [16]. Microsoft’s SAGE conducts large-scale application of whitebox fuzzing via symbolic execution to uncover security-critical bugs [43]. More recent work, such as Driller, augments fuzzing with selective symbolic execution to efficiently navigate deep execution paths that lead to vulnerabilities [15]. Despite these successes, symbolic execution remains challenged by path explosion and scalability issues [60], motivating research into hybrid approaches that combine symbolic reasoning with other techniques.

Algorithm 5: Anchored Random Walk

Input: *ICFG*: ICFG, *E*: Program entry, *L_{patch}*: Patch location (optional), *L_{manifest}*: Manifestation point
Output: *path*: a valid execution path

```

1 Function RandomWalk(ICFG, E, Lpatch, Lmanifest)
2   Dmanifest ← ComputeDistances(ICFG, Lmanifest)
3   if Lpatch is not null then
4     Dpatch ← ComputeDistances(ICFG, Lpatch)
5     path ← RecursiveWalk(E, to_patch, [])
6   else
7     path ← RecursiveWalk(E, to_manifest, [])
8   return path;

9 Function RecursiveWalk(curr, progress, path)
10  path.append(curr);
11  UpdateCallStack(curr);
12  if curr = Lpatch and progress = to_patch then
13    progress ← to_manifest
14  if curr = Lmanifest and progress = to_manifest then
15    return path
16  successors ← GetSuccessors(curr)
17  weights ← [];
18  totalWeight ← 0.0;
19  foreach s in successors do
20    if progress = to_patch then
21      dist ← Dpatch[s]
22    else
23      dist ← Dmanifest[s]
24    w ← 1.0/(dist + 1.0)
25    weights.append(w);
26    totalWeight ← totalWeight + w;
27  for i ← 1 to MAX_TRIES do
28    r ← Random(0, totalWeight)
29    next ← successors[successors.size() - 1]
30    for j ← 0 to weights.size() - 1 do
31      r ← r - weights[j]
32      if r ≤ 0 then
33        next ← successors[j]
34        break;
35    result ← RecursiveWalk(next, progress, path)
36    if result ≠ failure then
37      return result;
38  del curr in path
39  return failure;

```

3) *Automatic Exploit Generation*: Automatic Exploit Generation (AEG) is a field of security research focused on systematically creating inputs that trigger and exploit software vulnerabilities. Pioneering approaches in this area often leverage powerful program analysis techniques. For instance, symbolic execution, as exemplified by tools like KLEE, explores program paths by treating inputs as symbolic variables, allowing it to generate concrete test cases that satisfy specific path constraints and reach vulnerable code [16]. Another prominent approach is fuzzing, particularly hybrid methods like Driller, which augment traditional fuzzing with selective symbolic execution to navigate complex program logic and discover deep bugs [15]. Beyond these foundational methods, the AEG landscape has matured to include a wide array of specialized tools. Researchers have developed systems tailored to specific vulnerability classes, such as memory corruption bugs [17] or object injection flaws in web languages [18], [19]. This specialization also extends to the execution environment, with distinct tools developed for kernel-space vulnerabilities [20], web applications [21], and interpreters for high-level

languages like PHP and Python [46].

While these techniques have demonstrated success in specific contexts, their application to the broad and practical task of reproducing disclosed vulnerabilities faces several challenges. A primary issue is practicality and the high degree of manual effort required. Many powerful tools are not fully automated; symbolic executors often require custom-written harnesses to model program environments [16], and fuzzers may need carefully crafted initial inputs to be effective [15]. Furthermore, the high degree of specialization in existing AEG systems limits their generality, which is crucial for our goal of large-scale PoV reproduction. A tool designed for JavaScript logic bugs cannot be applied to C/C++ memory errors, and a kernel exploit generator operates with assumptions entirely incompatible with a web application context. This fragmentation means that no single, one-size-fits-all platform exists for reproducing the diverse range of vulnerabilities found in real-world software. Each new language, domain, or vulnerability class often requires a specialized approach, making automated, large-scale PoV generation for disclosed vulnerabilities challenging.

B. More Details on Technique

1) *Anchored Random Walk*: The anchored random walk is detailed in Algorithm 5. It takes the ICFG and the anchor nodes as input and returns a valid execution path connecting them.

The algorithm starts by invoking RandomWalk (line 1). It first pre-computes the shortest distance from every node to the manifestation anchor $L_{manifest}$ using a backward BFS, storing it in the distance map $D_{manifest}$ (line 2). In patch-guided mode, where the patch location L_{patch} is provided, it also pre-computes D_{patch} and starts the recursive walk with progress state to_{patch} (lines 4-5). In patch-free mode, where L_{patch} is null, the procedure skips the D_{patch} computation and starts the recursive walk directly with progress state $to_{manifest}$ (line 7).

The RecursiveWalk function (line 9) is the core of the pathfinding logic. It first adds the current node to the path and updates the call stack to handle function calls and returns (lines 10-11). The progress variable tracks the current target anchor. If L_{patch} is reached while the walk is targeting it, the progress is updated to target $L_{manifest}$ (lines 12-13). Note that in patch-free mode, this check is never satisfied because the walk starts with progress $to_{manifest}$ and never enters the to_{patch} state. If $L_{manifest}$ is reached while the walk is targeting it, the complete path is found and returned (lines 14-15).

Next, the algorithm retrieves all valid successor nodes (line 16). It then iterates through these successors to calculate a weight for each one (lines 19-26). The weight is calculated as the inverse of the successor’s distance to the current target anchor (line 24), ensuring that closer nodes receive higher weights. The total weight is also accumulated.

The algorithm then enters a loop to try exploring from the current node multiple times (line 27). In each trial, a random

number is generated within the range of the total weight (line 28). It then iterates through the successors, subtracting their weights from the random number until it falls below zero (lines 30-34). The successor that causes this is chosen for the next step, implementing a weighted random selection. A recursive call is made with the selected successor (line 35). If a valid path is found, it is returned immediately. If all attempts fail, the algorithm backtracks by removing the current node from the path and returning a failure signal (lines 38-39).

2) *LLM Prompt Template*: This section provides the specific templates used to interact with the Large Language Models (LLMs) in the POVGEN framework.

On-Demand Interprocedural Slicing. This prompt is used in on-demand interprocedural slicing to iteratively expand the scope of the code slice. The model is given an initial slice around the patch location and is asked to identify any user-defined functions for which it lacks context. This allows the system to retrieve the necessary code for those functions and incorporate them into an expanded, more comprehensive slice.

Question: Given the following vulnerable code slice <code-slice> where the vulnerability patch is <ground-truth patch>, you will need to identify the statements where the vulnerability is triggered (e.g., via a sanitizer). Please tell me the USER-DEFINED functions that you don’t know in this process, give me the function names. Please use the following template (plain text, do not add any format content) to answer without explanation: Functions not knowing about:
<function1>
<function2>
<CALLER_OF_function>
...

As indicated in the prompt, the model can also request information about the functions that call the patching function <CALLER_OF_function>. In such cases, the functions that call the existing functions in the slice can also be included.

Once the LLM provides the names of the necessary context functions, we query the SDG to perform interprocedural slicing that includes the code dependent on the patching statements within these newly identified functions.

Manifestation Localization Fine-Tuning. This prompt is used to fine-tune the manifestation localization model. The model is fine-tuned on pairs of code slices and their corresponding ground-truth patches. Its task is to identify the precise statement within the slice where the vulnerability manifests. This process creates a specialized model adept at pinpointing the true manifestation point, which serves as the target for the subsequent steps.

Question: Given the following vulnerable code slice <code slice>, where the vulnerability patch is <ground-truth patch>, now identify the statements in the slice where the vulnerability is triggered (e.g., via a sanitizer). If there is no such statements in this slice, please output “None”.
Response: <vulnerability manifestation statement>

Symbolic-Execution Knowledge Augmentation. These two prompts are used to fine-tune the neural-symbolic models. The goal is to augment the formal reasoning capabilities of a traditional symbolic execution engine and an SMT solver to two fine-tuned LLMs. The first prompt fine-tunes the *constraints extraction model* to derive logical constraints from a C program. The second prompt fine-tunes the *constraints solving model* to solve the constraints and generate a concrete test case (a PoV) that satisfies a given set of constraints.

Question: Extract and simplify the constraints to trigger the error for the following C program <Source Code>.
Response: <Constraints>

Question: Solve the constraints <Constraints> and generate a satisfying test case to trigger the error/vulnerability for the C program <Source Code>.
Response: <PoV>

LLM-based PoV Generation. These prompts are used in the final step of Step 3 to generate a PoV for the target vulnerability. The first prompt is directed at the fine-tuned *constraints extraction model*. It provides the model with a viable execution path, the vulnerability description, and the patch, asking it to produce the corresponding path constraints in SMT2 format. The second prompt takes these generated constraints and feeds them to the fine-tuned *constraints solving model*, which then produces the final Python script to generate the PoV.

Question: You are a symbolic execution engine. Given the execution path, vulnerability description, and the patch, please analyze the constraints on the execution path to trigger the vulnerability.
 Execution Path: <Execution Path>
 Vulnerability Description: <Vulnerability Description>
 Patch: <Patch>
 Please directly output the constraints in smt2 format.

Question: You are a symbolic execution engine. Given the execution path, vulnerability description, the patch, and the constraints to trigger the vulnerability, please solve the constraints and generate a satisfying PoV that can trigger the vulnerability.
 Execution Path: <Execution Path>
 Vulnerability Description: <Vulnerability Description>
 Patch: <Patch>
 Constraints: <Extracted SMT2 Constraints>
 Please directly output the python script to generate the PoV without explanation.

LLM Prompt Template for Baseline Comparison. To create a baseline that is directly comparable to our system’s methodology, the following prompt was used for the powerful LLM comparison in RQ2. It instructs the LLM to follow a multi-step workflow that conceptually parallels the manifestation localization, path exploration, and PoV generation steps of POVGEN.

Question: You are an expert cybersecurity researcher. Your task is to analyze the provided artifacts and generate a working Proof-of-Vulnerability (PoV) by following a systematic, multi-step process.
Artifacts Provided:
CVE Description: [Vulnerability Description Text]
Security Patch (Diff): [Patch/Diff Text]
Source Code: [Full Source Code of the Program]
Required Analysis Workflow:
Step 1: Vulnerability Manifestation Point Identification. Analyze the patch and the surrounding code. Based on the logic, identify the specific line of code where the vulnerability *manifests* (e.g., the out-of-bounds read, use-after-free). This may be different from the patch location itself.
Step 2: Execution Path Formulation. Describe a plausible execution path from a program entry point to the manifestation point identified in Step 1, ensuring the path passes through the code to be patched.
Step 3: Constraint Derivation. Based on the execution path from Step 2, derive the logical constraints on the input data required to trigger the vulnerability. Describe what must be true about the input values to force execution along this specific path.
Step 4: PoV Script Generation. Translate the constraints from Step 3 into a complete Python script that generates the binary PoV data to a file named `pov.bin`. The script must be self-contained and ready to execute to generate the PoV.

3) Data Augmentation Templates: To enhance the fine-tuning data for our neural-symbolic models, we introduce four templates that cover complex programming constructs often associated with vulnerabilities. These templates ensure the models are fine-tuned on scenarios that go beyond simple path constraints. The code for these templates is shown in Figure 4.

- **Loop with a Symbolic Bound.** This template injects the symbolic-loop motif. It features a for loop that iterates up to a symbolic integer n , with an accumulator variable conditionally updated inside the loop and an assertion on its final value. To satisfy the assertion, the model has to reason about the loop’s behavior in a generalized way rather than unrolling it, which exposes it to the kind of input-length reasoning that benchmark samples alone underrepresent.
- **Pointer and Symbolic Dereference.** This template injects the pointer-aliasing motif. It declares a symbolic integer x and a pointer p initialized to the address of x , with the final assertion checking a condition on $*p$. The model must recognize that $*p$ and x refer to the same symbolic value, exposing it to the basic pointer-indirection pattern that appears frequently in real-world code.
- **Symbolic Array and Nested Loops.** This template injects the indexed-access motif. It introduces a symbolic array and uses nested loops to compute a value based on pairs of array elements ($arr[i]$, $arr[j]$), with the final assertion on the resulting counter. The model is exposed to indexed memory access where both the array’s contents and the logic operating on them are symbolic.
- **Pointer Arithmetic with a Symbolic Offset.** This template injects the symbolic-offset motif underlying many buffer-related issues. It initializes a concrete array arr and declares a symbolic integer offset, with a pointer p calculated as $arr + offset$. The compound assertion checks both that the offset is within array bounds ($offset \geq 0 \ \&\& \ offset < size$) and that $*p$ satisfies a specific condition. The model is exposed to the multi-part constraints typical of out-of-bounds access reasoning.

In the four templates, op is a relational operator (e.g., $>$, $==$, $<=$) selected randomly, while val and $threshold$ are random integer values.

C. More Details on Results

1) *Illustrating Examples for POVGEN:* Figures 8, 9, and 10 provide a step-by-step illustration of how POVGEN generates a PoV for the vulnerability CVE-2022-23537, originally shown in Figure 2.

Figure 8 demonstrates Step 1, Vulnerability Manifestation Localization. Starting from the patch at line 2360 in `stun_msg.c`, POVGEN performs forward and backward slicing. It then uses an LLM to request context for the `decode_errcode_attr` function, expanding the slice. Finally, the fine-tuned manifestation localization model analyzes this slice to pinpoint the true manifestation point at line 1760, which is distant from the patch location.

Figure 9 shows the result of Step 2, Path-Sensitive Reachability Exploration. Using the program entry point (LLVM-FuzzerTestOneInput), the patch location (line 2360), and the manifestation point (line 1760) as anchors, POVGEN performs a guided random walk to identify a viable execution path that connects all the anchor points. This narrows the analysis from the entire project to a single, focused trace.

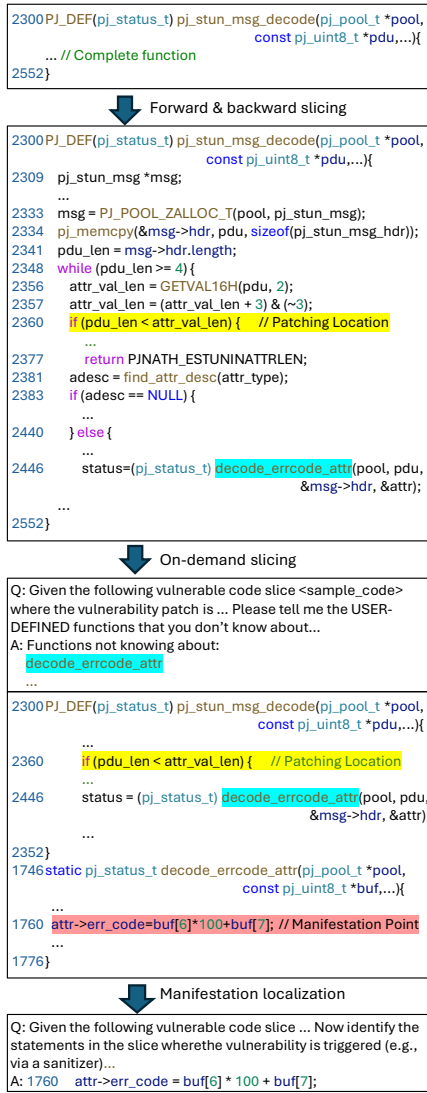


Fig. 8. Vulnerability manifestation localization for example in Figure 2.

Figure 10 illustrates the final step, Neural-Symbolic PoV Generation. The fine-tuned constraints extraction model takes the execution path from the previous step and derives the precise SMT2 constraints required to trigger the vulnerability. Subsequently, the fine-tuned constraints solving model uses these constraints to generate a concrete Python script that produces the working PoV.

2) *Robustness of POVGEN*: Figure 11 demonstrates the robustness of POVGEN’s pipeline, even when the initial manifestation localization is not perfectly accurate. In this example, the manifestation localization model in Step 1 predicted the manifestation point to be at line 250, while the ground truth was at line 243. However, because both lines reside within the same control-flow block, the execution path generated in Step 2 was still valid for triggering the vulnerability. Consequently, Step 3 was able to extract the correct constraints from this path and successfully generate a working PoV. This shows that POVGEN can tolerate minor inaccuracies in manifestation

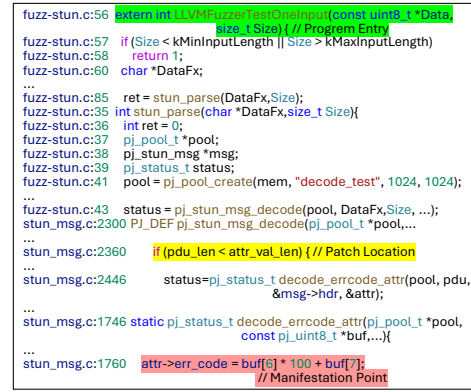


Fig. 9. Extracted execution path for the example in Figure 2.

point prediction, as long as the identified location lies on the correct vulnerable path.

3) *Failure Cases of Baselines*: Figures 12 and 13 illustrate why baseline LLM approaches fail on the same CVE-2022-23537 example, highlighting the necessity of POVGEN’s multi-step design.

Figure 12 shows the failure of direct, high-level prompting. A powerful commercial LLM (Gemini-2.5-Pro) was provided with the entire project’s source code and a prompt to generate a PoV. The model failed because it was overwhelmed by the vast and noisy context (Challenge 2) and could not identify the precise vulnerability manifestation point (Challenge 1). The resulting Python script is generic and fails to generate an input that triggers the vulnerability.

Figure 13 demonstrates the failure of using a non-fine-tuned model, even with a focused execution path. Here, a base model (Gemini-2.5-Flash) was given the correct, focused execution path derived from Step 2, thus overcoming the scaling challenge. However, it still failed because it lacked the specialized, formal reasoning capability to correctly extract and solve SMT constraints (Challenge 3). The generated constraints and the final PoV script are logically flawed and do not reproduce the bug, underscoring the critical role of fine-tuning for the neural-symbolic PoV generation step.

```
Q: You are a symbolic execution engine. Given the source code, vulnerability description, the patch, please generate a python script that can generate a PoV file named `pov.bin` ?
Source Code:
// _pjsip/tests/fuzz/fuzz-stun.c
1 *
2 * Copyright (C) 2008-2011 Teluu Inc. (http://www.teluu.com)
3 * Copyright (C) 2003-2008 Benny Prijono <benny@prijono.org>
*/
...
18 #include <stdio.h>
19 #include <stdint.h>
20 #include <stdlib.h>
...
32 pj_pool_factory *mem;
...
54 extern int LLVMFuzzerTestOneInput(const uint8_t *Data, size_t
Size)
... // 100k+ lines of code

Vulnerability Description:
...

Patch:
diff a/pjath/src/pjath/stun_msg.c
b/pjath/src/pjath/stun_msg.c
...

Please output the python script to generate PoV in the format without any explanation.
```

```
A:
def generate_pov():
    # STUN Message Header (20 bytes) Message Type: 0x0001
    # (Binding Request)
    msg_type = 0x0001
    # Malicious Attribute Header (4 bytes) Attribute Type:
    0x8FFF (An unknown, optional attribute type)
    attr_type = 0x8FFF
    # Attribute Length: 0x0004. This value causes the over-read.
    The parser will attempt to read 4 bytes starting from an out-
    of-bounds location.
    attr_length = 0x0004
    attribute = struct.pack('!HH', # Big-endian: unsigned short,
    unsigned short attr_type, attr_length)

    pov_data = header + attribute
    with open('pov.bin', 'wb') as f:
        f.write(pov_data)

if __name__ == '__main__':
    generate_pov()
```

Fig. 12. Direct PoV generation using Gemini-2.5-Pro fails on the example in Figure 2.

Fig. 11. An example where the predicted manifestation point is different from the ground-truth one but POVGEN still generates PoVs correctly.

Q: You are a symbolic execution engine. Given the execution path, vulnerability description, and the patch, please analyze the constraints on the execution path to trigger the vulnerability and generate a python script that can generate a PoV file named pov.bin.

Execution Path:
fuzz-stun.c:56 extern int LLVMFuzzerTestOneInput(const uint8_t *Data, size_t Size){
...
stun_msg.c:2360 if (pdu_len < attr_val_len) {
...
stun_msg.c:1760 attr->err_code = buf[6] * 100 + buf[7];
...
Vulnerability Description:
PJSIP is a free and open source multimedia communication library written in C language implementing standard based protocols such as SIP, SDP, RTP, STUN, TURN, and ICE. Buffer overread is possible when parsing a specially crafted STUN message with unknown attribute. The vulnerability affects applications that uses STUN including PJNATH and PJSUA-LIB. The patch is available as a commit in the master branch (2.13.1).

Patch:
2360c2360
< if (pdu_len < attr_val_len) {

> if (pdu_len < attr_val_len + ATTR_HDR_LEN) {
Please directly output the concrete constraints in smt2 format and the python script to generate PoV in the format without any explanation

A: Constraints:
; Must have at least 4 bytes for attribute header
(assert (>= pdu_len 4))
; Attribute header parsing
(assert (= attr1_type (bv2int (concat (select Data 20) (select Data 21)))))
(assert (= attr1_val_len_raw (bv2int (concat (select Data 22) (select Data 23)))))
; Padded length calculation (stun_msg.c:2357)
(assert (= attr1_val_len_padded (bvand (bvadd attr1_val_len_raw 3) (bvmul (bvnot 3) 1))))
; Vulnerability condition (pdu_len < attr_val_len_padded is FALSE, but pdu_len < attr_val_len_padded + 4 is TRUE)
(assert (>= pdu_len attr1_val_len_padded))
; Patched check:
(assert (< pdu_len (+ attr1_val_len_padded 4)))

PoV Script:
import struct
msg_type = 0x0101 # Binding Response (to make PJ_STUN_IS_REQUEST false)
msg_length = 13 # Length of attributes section (13 bytes)
magic_cookie = 0x2112A442
transaction_id = b'\x00' * 12 # Arbitrary transaction ID
attr_type = 0x0000 # Unknown attribute type
attr_val_len_raw = 10 # Raw attribute value length
stun_header = struct.pack("!HHII", msg_type, msg_length, magic_cookie, 0) + transaction_id[4:]
attribute_header = struct.pack("!HH", attr_type, attr_val_len_raw)
attribute_value = b'A' * attr_val_len_raw # Arbitrary value for the attribute
attribute_payload = attribute_header + attribute_value[:9]
pov_data = stun_header + attribute_payload # Combine header and attributes
with open("pov.bin", "wb") as f:
f.write(pov_data)

Fig. 13. Direct use of Gemini-2.5-flash for constraints and PoV generation without fine-tuning fails on the example in Figure 2.